

sl-lab 주문시스템 설계서

실증 랩

화면 설계서 5

API 설계서 15

데이터 설계서 6

버전 v1.0
작성일 2026-09-07
작성자

대외비

목차

화면 설계서 5건

- UIS-ORD-001
- UIS-ORD-002
- UIS-ORD-003
- UIS-ORD-004
- UIS-ORD-005

API 설계서 15건

- INF-ORD-001
- INF-ORD-002 회원 상세 조회
- INF-ORD-003
- INF-ORD-004
- INF-ORD-005
- INF-ORD-006
- INF-ORD-007
- INF-ORD-008
- INF-ORD-009
- INF-ORD-010
- INF-ORD-011
- INF-ORD-012
- INF-ORD-013
- INF-ORD-014
- INF-ORD-015

데이터 설계서 6건

- SCH-ORD-001 members
- SCH-ORD-002 order_delivery
- SCH-ORD-003 order_items
- SCH-ORD-004 orders
- SCH-ORD-005 products
- SCH-ORD-006 cart_items

개정 이력

버전	일자	내용	작성자
v1.0	2026-09-07	최초 작성	

화면 설계서

UIS-ORD-001

UIS-ORD-001: 주문 목록

근거 소스(권위): src/main/resources/templates/order/list.html (Thymeleaf 서버 렌더)

+

src/main/java/com/sm/lab/order/controller/OrderViewController.java. 스크린샷은 보조.

0. 화면 미리보기

주문 목록

회원 ID	M-0001	주문상태	전체	검색
주문번호	회원	상태	총액	주문일시
260817-0002	이도그 (M-0002)	CANCELED	450,000원	2026-08-17 09:40
260817-0003	박푸딩 (M-0003)	PLACED	129,000원	2026-08-17 09:00
260816-0003	김솔증 (M-0001)	PARTIAL_SHIPPED	579,000원	2026-08-16 16:05
260816-0001	이도그 (M-0002)	PAID	390,000원	2026-08-16 14:30
260815-0001	김솔증 (M-0001)	DONE	519,000원	2026-08-15 10:12

전체 5건

원 안 번호 = 「4. 위젯:액션」 표의 번호와 1:1.

1. 화면 목적

회원 ID·주문상태·조회기간 조건으로 주문 내역을 조회하는 목록 화면이다. 각 행의 주문번호를 클릭하면 해당 주문의 상세 화면(UIS-ORD-002)으로 이동한다.

2. 주요 작업 시나리오

시나리오: 주문 조회

- 「회원 ID」에 조회할 회원의 ID를 입력한다(선택 — 비우면 전체 회원 대상). (1)
- 「주문상태」 드롭다운에서 상태를 선택한다(선택 — 기본값 "전체"). (2)
- [반영: FUNC-order-001] 「시작일」·「종료일」 date 입력에 조회 기간을 입력한다(선택 — 둘 다 비우면 최근 30일, 하나만 비우면 그 값만 독립적으로 보정). (2-1)/(2-2)
- 「검색」 버튼을 클릭한다. 폼이 GET /order/list로 재요청되며 memberId·orderState·startDate·endDate 쿼리 파라미터(기간은 orderState와 AND 결합)로 필터링된 목록이 다시 렌더링된다. (3)
- 목록에서 원하는 주문의 주문번호 링크를 클릭하면 GET /order/{orderNo}로 이동해 주문 상세 화면(UIS-ORD-002)을 확인할 수 있다. (4~8)
- [반영: FUNC-order-001] 조회 기간이 역전(보정된 시작일 > 종료일)되면 검색 결과 대신 상단에

오류 배너("조회 실패: ...")가 표시된다(기존 cart/list.html의 checkoutError, product/detail.html의 addToCartError와 동일 하우스 패턴).

(별도의 등록/수정/삭제 시나리오 없음 — 이 화면은 조회·상세이동 전용)

3. 화면 구성 (블록)

블록	역할	주요 위젯	소스 근거
오류 배너	검색 실패(예: 조회기간 역전 400) 시 사유 표시	<div th:if="\${searchError != null}">	list.html:19-22 [반영: FUNC-order-001]
검색조건	회원 ID·주문상태·조회기간으로 주문 필터링	memberId 입력, selState 드롭다운, startDate/endDate date 입력([반영: FUNC-order-001]), btnSearch 버튼	list.html:23-33
주문목록 테이블	조회 결과 표시(주문번호/회원/상태/총액/주문일시) + 상세 이동 링크	주문번호 링크 5건(현재 페이지 기준)	list.html:35-45
총건수/빈결과 안내	조회 결과 총 건수 표시, 결과 0 건 시 안내 문구	전체 N건, 조회 결과 없음([반영: FUNC-order-001])	list.html:46-47

4. 위젯·액션

번호 = 「0. 화면 미리보기」 마커 번호와 1:1.

번호	위젯	타입	레이블	동작	연결 API(row)	결과
(1)	memberId	input(text)	회원 ID	검색 조건 입력 (placeholder 예시: M-0001)	—	폼 파라미터로 (3) 검색 시 전송
(2)	selState	select	주문 상태	검색 조건 선택(전체/PLACED/PAID/SHIPPED/PARTIAL_SHIPPED/CANCELLED/DONE)	—	폼 파라미터로 (3) 검색 시 전송
(2-1)	startDate	input(date)	시작 일	검색 조건 입력(미입력 시 화면은 자동채움 없음 — 서버가 기본값 보정)	—	폼 파라미터로 (3) 검색 시 전송, orderState와 AND. 마커 미부여(신규, [반영: FUNC-order-001], 스크린샷 재캡처 필요)

번호	위젯	타입	레이블	동작	연결 API(raw)	결과
(2-2)	endDate	input(date)	종료일	검색 조건 입력(미입력 시 화면은 자동채움 없음 — 서버가 기본값 보정)	—	폼 파라미터로 (3) 검색 시 전송, orderState와 AND. 마커 미부여(신규, [반영: FUNC-order-001], 스크린샷 재캡처 필요)
(3)	btnSearch	button(submit)	검색	폼(#searchForm, method=GET) 제출	GET /order/list	동일 화면 재렌더(필터 적용된 목록·총건수 갱신). 조회기간 역전 시 400 → 오류 배너 렌더(아래 참조)
(4)	주문번호 링크(예: 20260817-0002)	a	{주문번호}	해당 주문 상세로 이동	GET /order/{orderNo}	주문 상세 화면(UIS-ORD-002) 이동
(5)	주문번호 링크(예: 20260817-0001)	a	{주문번호}	해당 주문 상세로 이동	GET /order/{orderNo}	주문 상세 화면(UIS-ORD-002) 이동
(6)	주문번호 링크(예: 20260816-0002)	a	{주문번호}	해당 주문 상세로 이동	GET /order/{orderNo}	주문 상세 화면(UIS-ORD-002) 이동
(7)	주문번호 링크(예: 20260816-0001)	a	{주문번호}	해당 주문 상세로 이동	GET /order/{orderNo}	주문 상세 화면(UIS-ORD-002) 이동
(8)	주문번호 링크(예: 20260815-0001)	a	{주문번호}	해당 주문 상세로 이동	GET /order/{orderNo}	주문 상세 화면(UIS-ORD-002) 이동

(4)~(8)은 조회 결과 행 수만큼 반복 렌더되는 **동일 패턴의 링크**다(list.html:38, th:each="o : \${result['items']}"). 캡처 시점 조회 결과가 5건이라 5개로 나타났을 뿐, 표는 위젯 패턴을 설명한다.

5. 접근 권한·표시 조건

요소	표시 조건	근거
검색조건·목록·상세이동 링크 전체	조건 없음(항상 표시) — 소스에 auth: 슬롯·권한 분기 없음	list.html 전체

7. 데이터 출처·연결

- **연결 API(raw → INF):** GET /order/list(화면 자기갱신), GET /order/{orderNo}(상세 화면 이동)
—
둘 다 Thymeleaf 서버 렌더 라우트(kind:form)이며, docs/05_설계서/order/INF/의 /api/orders/* REST 엔드포인트(INF-ORD-003~007)와는 별개 계층이다. 화면은 OrderService를 서버 내부에서 직접 호출하므로 이 화면에서 INF로 치환되는 API 호출은 없다(정상 — INF 없음).
- **참조 테이블(SCH):** docs/05_설계서/order/SCH/(주문·회원·상품 테이블 — DB_order.md 참고)

8. 미확인 사항

- 목록 페이징 UI(다음/이전 페이지 이동 컨트롤)는 컨트롤러가 page 파라미터(defaultValue="1")를 받지만 list.html에는 페이지네이션 위젯이 렌더되지 않는다 — 현재 뷰에는 페이징 컨트롤 없음.
- 자동 마커 선택 스크립트(select_tab_widgets.py)는 이 화면에서 0~1건만 선택했다(주문번호 링크가 id/onclick 없이 순수 href만 가진 <a>라 필터 조건 id 있음 OR onclick 있음)을 충족하지 못했고, 검색 버튼은 기본 --toolbar-y 90 임계값(버튼 y=64)에 걸려 제외됨. 이 spec의 0.4번 마커는 DOM 스냅샷 전수(8건)를 에이전트가 수동으로 preview_widgets.json에 반영해 생성했다 — 누락 없음을 확인했으나, 순수 href 링크 화면에 대한 스크립트 개선이 필요하다.
- **[반영: FUNC-order-001]** 「0. 화면 미리보기」의 preview_annotated.png는 SR-205 반영 **이전** 캡처라 시작일/종료일 입력((2-1)/(2-2))과 오류 배너가 스크린샷에 없다 — 코드(소스=권위) 기준으로 본문만 갱신했다. 다음 /sl-recon-uis 재캡처 시 마커 번호를 갱신할 것.
- **[반영: FUNC-order-001, QA round2 권고2 = story ## 후속 추적(TODO) 1번]** 오류 배너가 뜬 4xx 응답에서도 result['items']가 항상 빈 리스트라 "조회 결과 없음" 안내가 오류 배너와 **동시에** 렌더된다(list.html:46-47의 th:if가 searchError와 무관). 사람이 "추적등록 후 진행"으로 수용한
기존 겹이며 다음 라운드에서 처리 예정.

UIS-ORD-002

UIS-ORD-002: 주문 상세

근거 소스(권위): src/main/resources/templates/order/detail.html (Thymeleaf 서버 렌더) +
src/main/java/com/sm/lab/order/controller/OrderViewController.java. 스크린샷은
보조.

0. 화면 미리보기

← 주문 목록

주문 상세 — 20260817-0002

회원	이도그 (M-0002)
상태	CANCELED
총액	450,000원
주문일시	2026-08-17 09:40

주문 상품

라인	SKU	상품명	수량	단가
1	SKU-1003	4K 모니터	1	450,000원

배송

배송번호	상태	송장	출고일시
배송 없음			

주문 취소

원 안 번호 = 「4. 위젯:액션」 표의 번호와 1:1.

1. 화면 목적

주문 1건(orderNo)의 상태·회원·총액·주문일시와 주문 상품 라인, 배송 이력을 한 화면에서 조회한다.
주문 상태와 무관하게 화면 하단의 「주문 취소」 버튼으로 즉시 취소 요청을 보낼 수 있다.

2. 주요 작업 시나리오

시나리오: 주문 상세 조회

- 주문 목록 화면(/order/list)에서 특정 주문을 클릭하거나, /order/{orderNo} 라우트로 직접 진입한다.
- 서버(OrderViewController#orderDetail)가 orderService.detail(orderNo)로 주문 1건을 조회해 화면에 렌더한다(GET 시점에 이미 데이터 확정 — 화면 내 별도 조회 액션 없음).
- 상단 표에서 회원·상태·총액·주문일시를 확인한다.
- 「주문 상품」 표에서 라인별 SKU·상품명·수량·단가를 확인한다.
- 「배송」 표에서 배송 이력(배송번호·상태·송장·출고일시)을 확인한다. 배송 이력이 없으면 "배송 없음" 1행이 표시된다.
- 목록으로 돌아가려면 상단 「← 주문 목록」 링크로 이동한다.

시나리오: 주문 취소

1. 화면 하단 「주문 취소」 버튼(㉠)을 클릭한다.
2. PATCH /api/orders/{orderNo}/cancel 호출 → 완료 시 location.reload()로 화면을 재조회해 갱신된 상태(CANCELED 등)를 반영한다.
3. 별도 확인(confirm) 다이얼로그 없이 즉시 요청이 전송된다(소스에 confirm 없음 — 8. 미확인 사항 참고).

3. 화면 구성 (블록)

블록	역할	주요 위젯	소스 근거
상단 네비게이션	목록 화면으로 복귀	← 주문 목록 링크(href)	detail.html:18
주문 요약 표	회원/상태/총액/주문일시 표시	정적 테이블(위젯 없음)	detail.html:21-26
주문 상품 그리드	라인별 SKU·상품명·수량·단가	반복 테이블(th:each)	detail.html:28-40
배송 이력 표	배송번호·상태·송장·출고일시	반복 테이블 + 빈 배열 시 안내행	detail.html:42-54
액션 영역	주문 취소	#btnCancel	detail.html:56-59

4. 위젯·액션

버튼/필드 + 동작 → API(raw) → 결과. 번호 = 「0. 화면 미리보기」 마커 번호와 1:1.

번호	위젯	타입	레이블	동작	연결 API(raw)	결과
(1)	#btnCancel	button	주문 취소	현재 주문 (orderNo) 취소 요청	PATCH /api/orders/{orderNo}/cancel	성공 시 location.reload()로 화면 재조회·상태 갱신

「← 주문 목록」 링크(href="/order/list", 정적 앵커)와 상단/상품/배송 표는 인터랙션 위젯(id/onclick)이

아니므로 select_tab_widgets.py 전수 목록에 포함되지 않는다 — 3번 「화면 구성」에 별도 기록.

5. 접근 권한·표시 조건

요소	표시 조건	근거
화면 전체	권한 분기 없음 — /order/{orderNo} 진입 시 무조건 렌더	OrderViewController.java:31-35
「주문 취소」 버튼	항상 노출 — 현재 주문 상태(CANCELED 등)에 따른 disabled 분기 없음	detail.html:56-59

6. 팝업·연계 화면

팝업 없음(섹션 생략 대상이나 이동 대상만 기록).

트리거 위젯	팝업/연계 화면	연결 API/화면 (raw → INF)	용도
← 주문 목록 링크	주문 목록 화면	GET /order/list ← 화면 이동(INF 없음)	목록으로 복귀

7. 데이터 출처·연결

- **연결 API(raw → INF):**
- GET /order/{orderNo} — 화면 자체 라우트(서버 렌더 진입점). INF 대상 아님(view 반환, JSON API 아님).
- PATCH /api/orders/{orderNo}/cancel → INF-ORD-006
- **참조 테이블(SCH):** ORDERS, MEMBERS, ORDER_ITEMS, PRODUCTS, ORDER_DELIVERY (INF-ORD-004/006 기준 추정 — 화면 자체는 컨트롤러가 orderService.detail() 1회 호출로 전 데이터를 서버 렌더하며, 별도 REST API 조회는 없음)

8. 미확인 사항

- 「주문 취소」 버튼에 확인(confirm) 다이얼로그가 없어, 클릭 즉시 PATCH 요청이 전송된다 — 의도된 UX인지 확인 필요.
- 화면 소스(detail.html)에는 주문 상태(CANCELED 등)에 따라 「주문 취소」 버튼을 비활성화하는 분기가 없다 — 이미 취소된 주문에서도 버튼이 그대로 노출된다(실제 캡처 상태=CANCELED 화면에서도 버튼 활성 확인). 서버측(/api/orders/{orderNo}/cancel) 상태 검증 여부는 이 화면 소스만으로는 미확인.
- GET /order/{orderNo} 자체는 화면 라우트라 INF 매칭 대상이 아니며, 실제 데이터 조회 API(orderService.detail() 내부 구현)는 이 슬라이스 범위 밖.

UIS-ORD-003

UIS-ORD-003: 상품 목록

근거 소스(권위): src/main/resources/templates/product/list.html (Thymeleaf 서버 렌더) +

src/main/java/com/sm/lab/order/controller/ProductViewController.java. 스크린샷은 보조.

SR-201로 신규 구현된 화면(FUNC-order-009, Done).

0. 화면 미리보기

상품 목록

상품명 키워드 검색

SKU	상품명	가격	재고	상태
SKU-1001	판매중 데스크	390,000원	12	판매중
SKU-1002	개식 키보드	129,000원	40	판매중
SKU-1003	라 미니탑	450,000원	7	판매중

원 안 번호 = 「4. 위젯·액션」 표의 번호와 1:1.

1. 화면 목적

판매중 상품 목록을 상품명 키워드로 검색·조회하는 화면이다. 각 행의 상품명 링크를 클릭하면 해당 SKU의 상품 상세 화면(/product/{sku}, FUNC-order-010)으로 이동한다.

2. 주요 작업 시나리오

시나리오: 상품 조회

- 「상품명」 입력란에 검색어를 입력한다(선택 — 비우면 전체 판매중 상품 대상). (1)
- 「검색」 버튼을 클릭한다. 폼(#searchForm, method=GET)이 GET /product/list?keyword=...로 재요청되며, 서버가 ProductService.list(keyword)로 상품명 부분일치(LIKE) 조회를 수행해 동일 화면을 재렌더한다. (2)
- 조회 결과 목록에서 원하는 상품의 상품명 링크를 클릭하면 GET /product/{sku}로 이동해 상품 상세 화면을 확인할 수 있다. (3~5)

(별도의 등록/수정/삭제 시나리오 없음 — 이 화면은 조회·상세이동 전용 카탈로그 화면이다.)

3. 화면 구성 (블록)

블록	역할	주요 위젯	소스 근거
검색조건	상품명 키워드로 목록 필터링	keyword 입력, btnSearch 버튼	list.html:24-28
상품목록 테이블	조회 결과 표시(SKU/상품명/가격/재고/상태) + 상세 이동 링크	상품명 링크 N건(현재 페이지 기준)	list.html:29-42
빈 목록 안내	조회 결과 0건일 때 안내 문구	"표시할 상품이 없습니다"	list.html:43

4. 위젯.액션

번호 = 「0. 화면 미리보기」 마커 번호와 1:1.

번호	위젯	타입	레이블	동작	연결 API(raw)	결과
(1)	keyword	input(text)	상품명	검색 조건 입력(부분일치 검색어)	—	폼 파라미터로 (2) 검색 시 전송
(2)	btnSearch	button(submit)	검색	폼(#searchForm, method=GET) 제출	GET /product/list	동일 화면 재렌더(keyword LIKE 필터 적용된 목록 갱신)
(3)	상품명 링크(예: 스탠딩 데스크, SKU-1001)	a	{상품명}	해당 SKU 상세로 이동	GET /product/{sku}	상품 상세 화면(FUNC-order-010) 이동
(4)	상품명 링크(예: 기계식 키보드, SKU-1002)	a	{상품명}	해당 SKU 상세로 이동	GET /product/{sku}	상품 상세 화면(FUNC-order-010) 이동
(5)	상품명 링크(예: 4K 모니터, SKU-1003)	a	{상품명}	해당 SKU 상세로 이동	GET /product/{sku}	상품 상세 화면(FUNC-order-010) 이동

(3)~(5)는 조회 결과 행 수만큼 반복 렌더되는 **동일 패턴의 링크**다(list.html:34-40, th:each="p : \${products}"). 캡처 시점 조회 결과가 3건이라 3개로 나타났을 뿐, 표는 위젯 **패턴**을 설명한다. 상태 컬럼(판매중/품질)은 p.stockQty == 0 여부로 클라이언트 개입 없이 서버에서 계산되는 표시 전용 값(액션 없음, 마커 대상 아님).

5. 접근 권한.표시 조건

요소	표시 조건	근거
검색조건·목록·상세이동 링크 전체	조건 없음(항상 표시) — 소스에 auth: 슬롯·권한 분기 없음	list.html 전체, ProductViewController.java:32-37

7. 데이터 출처·연결

- **연결 API(raw → INF):** GET /product/list(화면 자기갱신), GET /product/{sku}(상세 화면 이동) — 둘 다 Thymeleaf 서버 렌더 라우트(kind:form)이며, 이 화면에서 INF로 치환되는 API 호출은 없다(정상 — INF 없음).
- **데이터 원천(서버사이드, api_hints 아님):** 이 화면은 브라우저에서 GET /api/products를 호출하지 않는다 — ProductViewController.productList()가 ProductService를 서버 내부에서 직접 주입·호출해 렌더링 데이터를 만든다(ProductViewController.java:32-37). 다만 그 내부 로직(판매중 상시필터 + keyword 부분일치, ProductService.list(keyword))은 [[INF-ORD-008]] (GET /api/products)와 **동일한 서비스 메서드를 공유**하므로, INF-ORD-008을 이 화면의 데이터 출처(사양상 원천 참고)로 명시한다. 두 계층은 진입 경로가 다를 뿐(REST vs 서버렌더) 조회 규칙은 같다 — INF-ORD-008의 screens:에는 이 화면을 실제 호출자로 등재하지 않는다(호출 사실이 없으므로).
- **참조 테이블(SCH):** [[SCH-ORD-005]] (PRODUCTS) — [[INF-ORD-008]] 참조 테이블과 동일.

8. 미확인 사항

- 목록 페이징 UI(다음 페이지 등)는 뷰·컨트롤러 어디에도 존재하지 않는다 — 전체(또는 keyword 필터) 결과가 페이징 없이 한 번에 렌더된다.
- 자동 마커 선택 스크립트(select_tab_widgets.py, 기본 --toolbar-y 90)는 이 화면에서 상품명 링크(순수 href만 가진 <a>, id/onclick 없음)를 필터 조건 id 있음 OR onclick 있음 미충족으로 제외했고, keyword 입력(input 태그)도 대상 태그(button/a)가 아니라 제외했다. --toolbar-y 30으로 낮춰 btnSearch(y=64)만 추가 확보(1건) 후, 나머지 4건(입력 1 + 링크 3)은 DOM 스냅샷 전수를 에이전트가 수동으로 preview_widgets.json에 반영해 생성했다(UIS-ORD-001과 동일 사례) — 누락 없음을 확인했으나, 순수 href 링크·input 태그 화면에 대한 스크립트 개선이 필요하다.

UIS-ORD-004

[변경: SR-202] 2026-08-22

[변경: SR-202] 2026-08-23 — 담기 폼 반영(FUNC-order-010 r6~r7 구현 실측)

UIS-ORD-004: 상품 상세

근거 소스(권위): src/main/resources/templates/product/detail.html (Thymeleaf 서버 렌더) +

src/main/java/com/sm/lab/order/controller/ProductViewController.java +

src/main/java/com/sm/lab/order/service/CartService.java. 스크린샷은 보조.

0. 화면 미리보기

[← 목록으로](#)

상품 상세 — SKU-1001

SKU	SKU-1001
상품명	스탠딩 데스크
가격	390,000원
재고	12
상태	판매중

회원 김철중 (M-0001) 수량

이 화면은 select_tab_widgets.py 기준 마킹 대상 인터랙션 위젯(id-onclick 보유 button/a) 0건이다

(재검증 완료). 담기 폼의 select/input/button은 실재하지만 소스에 id-onclick 속성이 없어(name 속성만 사용) 결정론 스캐너의 선택 대상이 아니다 — **결함이 아니라 소스 실측**이며,

아래 「4. 위젯.액션」에 소스 근거로 전수 기록했다. 유일한 링크(← 목록으로)는 상단 공통영역이라

3번 「화면 구성」에 기록. 따라서 마커 번호 없음, annotate 생략.

1. 화면 목적

상품 1건(sku)의 SKU·상품명·가격·재고·상태를 표시하고, 재고가 있으면 같은 화면에서 회원을 선택해 장바구니에 담을 수 있다(SR-202). 담기는 서버 렌더 POST(PRG)로 처리되며, 담기 자체는 재고를 차감하지 않는다(보관 전용). 존재하지 않는 SKU로 접근하면 예외 페이지 대신 "상품을 찾을 수 없습니다" 안내 문구를 보여준다.

2. 주요 작업 시나리오

시나리오: 상품 상세 조회 (정상 SKU)

1. 상품 목록 화면(/product/list, UIS-ORD-003)에서 상품명 클릭하거나, /product/{sku} 라우트로 직접 진입한다.
2. 서버(ProductViewController#productDetail → loadProduct)가 productService.get(sku)를 호출해 상품 1건을 조회한다(GET 시점에 이미 데이터 확정 — 화면 내 별도 조회 액션 없음).
3. 표에서 SKU·상품명·가격(#,###원 포맷)·재고·상태(재고 0이면 "품절", 그 외 "판매중")를 확인한다.
4. 목록으로 돌아가려면 상단 「← 목록으로」 링크로 이동한다.

시나리오: 존재하지 않는 SKU로 접근

1. 등록되지 않은 sku로 /product/{sku}에 접근한다.
2. productService.get(sku)가 던지는 404(ResponseStatusException)를 loadProduct가 흡수하고 product=null로 모델에 담아 같은 뷰(product/detail)를 그대로 반환한다(예외 스택 트레이스 노출 없음).
3. 화면에는 "상품을 찾을 수 없습니다" 안내 문구만 표시된다(상세 표·담기 폼 없음).
4. 「← 목록으로」 링크는 이 상태에서도 동일하게 노출되어 목록으로 복귀할 수 있다.

시나리오: 장바구니 담기 (재고 있음, SR-202 실측)

1. 재고(stockQty) > 0인 상품 상세 화면 하단의 담기 폼에서 회원을 선택한다(memberService.list())가 채운 기존 회원 목록 선택).
2. 수량을 입력한다(입력값 기본 1, min="1" — 클라이언트 힌트일 뿐 서버가 실검증한다).
3. [담기] 버튼을 클릭하면 POST /product/{sku}/cart {memberId, qty}가 제출된다. 컨트롤러는 회원·수량·재고 판정을 직접 수행하지 않고 CartService.addItem(memberId, sku, qty)를 **자바 메시드로 직접 호출**한다(HTTP 아님, REST POST /api/cart/items([[INF-ORD-010]])와는 별도 경로 — 같은 서비스 계층만 공유).
4. 같은 회원·같은 상품 재담기는 CartService가 DB 원자 UPSERT로 수량을 합산한다(D2, CART_ITEMS PK(member_id, sku)).
5. **성공**: 서버가 redirect:/product/{sku}로 302 리다이렉트하며 RedirectAttributes 플래시에 addToCartSuccess=true·addedMemberId를 담는다. 이어지는 GET 렌더에서 "장바구니에 담았습니다. 장바구니로 이동" 문구 + /cart?memberId=... 링크(UIS-ORD-005, 채번만)가 표시된다. 플래시는 1회성이라 그 다음 새로고침(F5)에서는 문구가 사라진다.
6. **거부**(400 수량<1 / 404 회원·상품 없음 / 409 재고초과·품절): 동일하게 302 리다이렉트 + addToCartError=<CartService 사유 메시지> 플래시 → 재렌더 시 사유 문구만 표시(재검증 로직 없이 서비스 메시지 그대로).
7. 두 경우 모두 F5(새로고침)는 리다이렉트된 **GET**을 재요청할 뿐이라 CartService.addItem이 중복 호출되지 않는다(PRG — QA r6→r7 결정, 아래 §8).

시나리오: 품절 상품

1. 재고(stockQty) == 0인 상품 상세 화면에서는 담기 폼 자체가 렌더되지 않고, "품절 상품은 담을 수 없습니다" 안내만 표시된다(폼을 통한 우회 제출은 불가 — 폼이 아예 없음).

3. 화면 구성 (블록)

블록	역할	주요 위젯	소스 근거
상단 네비게이션	목록 화면으로 복귀	← 목록으로 링크(href, 정상/미존재 양쪽 공통)	detail.html :20
상품 상세 표 (정상 SKU)	SKU·상품명·가격·재고·상태 5행 표시	정적 테이블(위젯 없음)	detail.html :29-37
안내 영역 (미존재 SKU)	"상품을 찾을 수 없습니다" 문구	정적 텍스트	detail.html :23-26
담기 성공 안내	담기 완료 확인 + 장바구니 이동 링크 (PRG 플래시 1회성)	안내 텍스트 + /cart? memberId=... 링크	detail.html :41-45
담기 거부 사유 안내	400/404/409 사유 메시지 표시 (PRG 플래시 1회성)	안내 텍스트	detail.html :49-51
장바구니 담기 폼 (재고 있음)	회원 선택 + 수량 입력 후 담기	회원 선택 셀렉트, 수량 input(기본 1/min1), [담기] 버튼	detail.html :54-65
품절 안내 (재고 0)	담기 폼 대신 안내 문구, 품 미노출	정적 텍스트	detail.html :68-70

4. 위젯·액션

소스 실측: 담기 폼의 select/input/button은 id-onclick 속성이 없어(name만 사용) select_tab_widgets.py 전수 스캔에서 마커 대상이 되지 않는다(0건, 결함 아님 — 위 0번 참고).

아래 표는 소스(detail.html)를 직접 읽어 도출한 **전수** 기록이다. 마커 번호는 없다.

위젯	타입	레이블	동작	연결 API(raw)	결과
회원 선택 (name="memberId")	select	회원	members(기존 회원 목록) 중 선택 — 옵션 0개(회원 0명)여도 폼은 렌더됨	—	선택값 = memberId, 제출 시 ""이면 CartService 404
수량 입력 (name="qty")	input(number, min=1, value=1)	수량	서버가 @RequestParam(defaultValue="1") 로도 기본 1 고정(이중 방어). min=1은 클라이언트 힌트일 뿐, 1 미만 제출은 서버(CartService)가 400으로 거부	—	입력값 = qty
[담기] 버튼	button(submit)	담기	폼 제출 → POST /product/{sku}/cart	POST /product	성공: 302 redirect + flash(성공 문구/장바구니 링크). 거부: 302 redirect + flash(사유).

위젯	타입	레이블	동작	연결 API(rw)	결과
				ct/{sku}/cart	5xx: 그대로 rethrow(흡수 안 함)
담기 성공 링크 (장바구니로 이동)	a	장바구니로 이동	성공 플래시가 있을 때만 렌더, 클릭 시 이동	GET /cart	UIS-ORD-005(장바구니, 채번만·화면 본문 미저작)로 이동
거부 사유 텍스트	text(조건부)	—	addToCartError != null일 때만 노출, CartService가 던진 <code>ResponseStatusException.reason</code> 을 그대로 출력(재검증·가공 없음)	—	400/404/409 사유 메시지 표시
품절 안내 텍스트	text(조건부)	—	stockQty == 0일 때만 노출(품 자체가 함께 미노출)	—	"품절 상품은 담을 수 없습니다"

5. 접근 권한·표시 조건

요소	표시 조건	근거
화면 전체 (GET/POST)	권한 분기 없음 — /product/{sku} GET·/product/{sku}/cart POST 모두 무조건 처리	ProductViewController.java:56-96
상세 표(5행)	product != null(정상 조회된 SKU)일 때만 렌더	detail.html:28
"상품을 찾을 수 없습니다" 문구	product == null(404 흡수)일 때만 렌더 — POST에서도 이 경우 addItem 호출 없이 즉시 이 문구로 응답(조기 반환)	detail.html:23, ProductViewController.java:78-82
담기 성공/거부 안내	각각 addToCartSuccess/addToCartError != null — PRG 플래시로 직후 GET 1회만 채워짐(그 다음 새로고침엔 사라짐)	detail.html:41,49
장바구니 담기 폼	product != null && product.stockQty > 0일 때만 렌더	detail.html:54
품절 안내	product != null && product.stockQty == 0일 때만 렌더(품과 상호배타)	detail.html:68
「← 목록으로」 링크	위 모든 분기와 무관하게 항상 노출	detail.html:20

6. 팝업·연계 화면

팝업 없음(화면 이동 대상만 기록).

트리거 위젯	팝업/연계 화면	연결 API/화면 (raw → INF)	용도
← 목록으로 링크	상품 목록 화면(UIS-ORD-003)	GET /product/list ← 화면 이동(INF 없음)	목록으로 복귀
담기 성공 링크(장바구니로 이동)	장바구니 화면(UIS-ORD-005, 채번만 — 화면 본문 미저작)	GET /cart ← 화면 이동(INF 없음)	방금 담은 장바구니 확인

7. 데이터 출처·연결

- **연결 API(raw → INF):**
- GET /product/{sku} — 화면 자체 라우트(서버 렌더 진입점). INF 대상 아님(view 반환, JSON API 아님). 내부적으로 ProductService.get(sku)를 **자바 메서드로 직접 호출**한다(HTTP 호출 아님) — 이 메서드는 REST API 컨트롤러([[INF-ORD-009]] GET /api/products/{sku})와 **동일 서비스 인스턴스를 공유**하지만, 상세 화면은 그 API를 HTTP로 부르지 않고 같은 서비스 계층을 재사용한다.
- POST /product/{sku}/cart — 담기 품 제출(서버 렌더 PRG). INF 대상 아님(리다이렉트 응답, JSON API 아님). 내부적으로 CartService.addItem(memberId, sku, qty)를 **자바 메서드로 직접 호출**한다(HTTP 호출 아님) — REST API인 [\[INF-ORD-010\]](#)와 **동일 CartService 인스턴스를 공유**하지만, 이 화면은 그 REST API를 직접 호출하지 않는다(서버사이드 직접 호출로 대체 — 명시적으로 구분해 둔다).
- GET /product/list — 목록 화면 이동. INF 대상 아님.
- GET /cart — 담기 성공 후 이동 링크(장바구니 화면). INF 대상 아님.
- **참조 테이블(SCH):**
- PRODUCTS ([[SCH-ORD-005]], INF-ORD-009 기준) — ProductService.get(sku) 경유.
- CART_ITEMS — CartService.addItem이 CartDao.upsertMergeQty로 갱신(REST 측 [[INF-ORD-010]]과 동일 테이블). **이 화면 기준 SCH 문서는 아직 없음** — 아래 §8 참조.
- MEMBERS(간접) — 담기 품의 회원 선택이 memberService.list()로 조회.

8. 미확인 사항

- **미존재 SKU 응답 코드 불일치 (SR-201 QA 결정 수용, r7까지 유지):** 이 화면은 존재하지 않는 SKU에 대해 **HTTP 200 + "상품을 찾을 수 없습니다"** 안내 문구를 렌더한다(GET·POST 조기 반환 양쪽 동일). 반면 같은 자원을 다루는 API [\[INF-ORD-009\]](#)는 **404**를 반환한다 — 두 계층의 "미존재" 의미가 일부러 다르게 설계됐다. STORY-FUNC-order-010 QA(r1 CONCERNS 권고3, r2 재게이트 PASS 시 잔존·수용, r7 재게이트에서도 잔존·수용으로 재확인)에서 "AC가 상태 코드를 규정하지 않고, 화면단은 사용자 안내가 목적이라 200 렌더가 화면 테스트·구현상 더 단순하다"는 이유로 **수용(waive)** 결정됨 — 결함이 아니라 화면/API 계층 간 의도된 차이로 기록. (근거: docs/00_FUNC/stories/STORY-FUNC-order-010.md QA Gate r1 권고3 / r7 잔존 권고1)
- **등록일 미표시 (SR-201 의사결정 D1, 변동 없음):** 요구사항 원문은 등록일 표시를 언급하나, PRODUCTS 테이블에 등록일 컬럼이 없어(DB 실측: sku-product_name-price-stock_qty-sale_yn 5개뿐) 이번 범위에서 **제외**했다. "DB 변경 없음" 원칙 유지가 이유이며, 등록일은 상품 등록

기능이 도입되는 추후 SR에서 컬럼과 함께 재검토한다. (근거:

docs/변경관리/SR-201/inputs/_decisions.md D1)

- F5 중복 담기 방지 — PRG 채택 근거 (SR-202, QA r6→r7):** r6 QA는 최초 구현이 담기 성공/거부를 같은 뷰(product/detail)로 **재렌더**(200, redirect 없음)해 F5 재제출 시 cartService.addItem이 2회 호출되고 수량이 조용히 누적됨을 프로브로 실측했다(CART_ITEMS는 PK UPSERT 합산이라 행 중복은 없었으나 수량 누적은 사용자 의도와 무관하게 발생). 기존 CartViewController(수량변경·삭제)가 이미 redirect:/cart?memberId=... 관례를 쓰고 있어 이 화면만 예외였던 점도 지적됨 (medium 회귀/UX, 권고2). r7에서 성공·거부 양쪽 모두 RedirectAttributes 플래시 + redirect:/product/{sku} (PRG)로 전환 — 재제출 대상이 POST가 아니라 멍등 GET이 되어 중복 호출이 구조적으로 제거됐다. r7 QA 재게이트가 프로브로 재확인: 성공/거부 각각 addItem 총 호출 1회, F5 후 플래시 소멸 → **PASS**. (근거: docs/00_FUNC/stories/STORY-FUNC-order-010.md QA Gate r6 CONCERNS 권고2 / r7 재게이트 "권고2(PRG) — 해소")
- CART_ITEMS의 SCH/추가 INF 문서 미생성(7-N 원칙, D5):** 담기 폼은 구현·QA 완료됐지만, 이 화면이 경유하는 CART_ITEMS 테이블의 SCH 문서와 POST/GET/PATCH/DELETE /api/cart/... 4종(D3)의 INF 중 POST /api/cart/items([[INF-ORD-010]])만 이미 존재하고 나머지는 아직 /s1-recon-inf·/s1-recon-sch가 소스에서 역생성하지 않았다. 이 화면 자체는 REST가 아니라 CartService를 직접 호출하므로 INF 신설 대상은 아니나, §7의 CART_ITEMS SCH 링크가 비어 있는 이유는 이 잔여 recon 미실행 때문이다.
- 경계 케이스(잔존, 변동 없음):** 경로 변수 경계 케이스(/product/ 빈 sku, / 포함 sku)는 이 화면의 친화 안내(200+문구) 대상이 아니라 Spring 기본 404로 빠진다(스택 트레이스 노출 없음) — 실 SKU 형식(SKU-####)에는 해당 없어 조치 불요(QA r2 잔존 권고2, r7까지 무변화). 회원 0명이면 옵션 0개인 빈 셀렉트 + 살아있는 [담기] 버튼이 렌더되고 제출 시 404("회원 없음")가 표시되는 것, 폼 우회 요청에서 memberId 누락 시 스프링 기본 400이 되는 것도 QA가 실측했으나 랩 데이터상 실사고 없어 인지 사항으로만 유지된다(QA r6 권고5·6, r7까지 무변화).

UIS-ORD-005

UIS-ORD-005: 장바구니

[변경: SR-203] 2026-08-23 — 체크아웃(주문하기) 반영 완료(구현 완료 후 재캡처 기준 재생성)

근거 소스(권위): src/main/resources/templates/cart/list.html(Thymeleaf 서버 렌더) + src/main/java/com/sm/lab/order/controller/CartViewController.java. 스크린샷은 보조.

0. 화면 미리보기

장바구니

회원

상품명	단가	수량	품목합계	관리
기계식 키보드	129,000원	2 변경	258,000원	삭제

총 합계 258,000원

[상품 목록으로](#)

재캡처 시점(FUNC-order-012 구현 완료 후, 장바구니 1품목 상태) 위젯 7개 중

#selMember·#btnSelect·

#btnCheckout 3개는 id를 갖지만, 행별 수량 input·[변경]·[삭제] 버튼 4개는 여전히 id-onclick이

없는 순수 HTML <form method="post"><button type="submit"> 네이티브 제출 방식이다 (JS 바인딩 없음).

select_tab_widgets.py(id/onclick 보유 위젯만 선택) 기준으로는 3개만 걸러 전수 조건을 만족하지

못하므로, 이번에도 자동 마커·annotate를 생략하고 **DOM 스냅샷의 위젯 번호(n, 1~7)**를 그대로

참조 번호로 사용해 전수 문서화한다(위젯 개수 7 = 아래 4번 표 행 수, 누락 없음).

1. 화면 목적

회원별로 서버에 보관된 장바구니(품목·수량·합계)를 조회하고, 그 자리에서 수량을 바꾸거나 품목을 삭제하며, 담긴 품목을 그대로 주문으로 전환(체크아웃)한다. 로그인 없이 상단 선택으로 조회 대상 회원을 고른다. 담기 자체(신규 추가)는 이 화면의 책임이 아니다 — 상품 상세 화면(UIS-ORD-004)에서 담고, 여기서는 이미 담긴 품목만 다루고 주문으로 마무리한다.

2. 주요 작업 시나리오

시나리오: 회원별 장바구니 조회

1. /cart 또는 /cart?memberId=로 진입한다. memberId를 안 주면 서버가 MemberService.list()의 첫 회원을 기본 선택한다(CartViewController#cart).
2. 상단 회원 선택(1)에서 다른 회원을 고르면 onchange="this.form.submit()"으로 즉시 재조회된다(GET /cart?memberId=...). [조회] 버튼(2)은 선택을 안 바꾼 상태에서 품을 다시 제출하고 싶을 때의 보조 수단이다(JS로 이미 자동 제출되므로 평소엔 클릭할 일이 없다).
3. 품목 표에서 상품명·단가·수량·품목합계를 확인하고, 표 아래 총합계(모든 품목의 lineTotal 합)를 확인한다.

시나리오: 품목 수량 변경

1. 해당 행의 수량 입력칸(3)을 원하는 값으로 고친다(min="1").
2. 같은 행의 [변경] 버튼(4)을 누르면 POST /cart/items/{sku}/update가 memberId(hidden)와 qty를 실어 제출된다.
3. 서버(CartViewController#updateQty → CartService.updateQty)가 처리 후 redirect:/cart?memberId=...로 돌아온다(PRG 패턴 — 새로고침 시 중복 제출 없음).
4. 재조회된 화면에서 바뀐 수량·품목합계·총합계를 확인한다. qty<1이면 400, 재고 초과면 409로 거부되며 두 경우 모두 사유 문구는 응답 본문에 담기지만 이 화면 자체로 재렌더되지 않고 오류 페이지로 이탈한다(8번 참조 — 알려진 한계).

시나리오: 품목 삭제

1. 해당 행의 [삭제] 버튼(5)을 누른다. 수량을 0으로 낮춰 사라지게 하는 방식이 아니라 삭제는 반드시 이 버튼으로만 한다(SR-202 D4 — 수량 1 미만은 거부하고 삭제 유도는 별도 버튼으로).
2. POST /cart/items/{sku}/delete가 memberId(hidden)를 실어 제출된다.
3. 서버(CartViewController#delete → CartService.delete)가 처리 후 redirect:/cart?memberId=....
4. 재조회된 화면에서 해당 행이 사라지고 총합계가 다시 계산된다. 마지막 품목을 지우면 빈 상태로 전환된다.

시나리오: 빈 장바구니 → 상품 목록으로 이동

1. 선택된 회원의 장바구니에 품목이 없으면 표·총합계·[주문하기] 버튼 대신 "장바구니가 비어 있습니다" 문구와 "상품 목록으로" 링크만 노출된다(SR-202 확정 scr_empty_state; 체크아웃 품은 D6 요구사항대로 품목이 있을 때만 렌더되므로 빈 상태에서는 애초에 노출되지 않는다).
2. 링크를 클릭하면 GET /product/list(UIS-ORD-003)로 이동해 담을 상품을 고른다.
3. 품목이 있는 상태에서는 표·총합계·[주문하기] 버튼 아래에 같은 "상품 목록으로" 링크(7)가 별도로 한 번 더 노출된다(list.html:88) — 추가 담기를 위해 목록으로 돌아가는 용도로 항상 존재.

시나리오: 체크아웃(주문하기)

1. 품목이 있는 상태에서만 [주문하기] 버튼(6)이 노출된다(list.html:77, th:if="\${!lists.isEmpty(cart['items'])}"). 빈 장바구니면 이 품 자체가 렌더되지 않아 버튼을 누를 수 없다.

2. 버튼(6)을 누르면 POST /cart/checkout가 memberId(hidden)를 실어 제출된다(PRG 패턴).
3. 서버(CartViewController#checkout → CartService#checkout)가 해당 회원 장바구니를 SELECT ... FOR UPDATE로 잠근 뒤 전 품목을 사전 스윙(상품없음/판매중지/재고부족)한다. 문제 없으면 OrderService.create로 위임해 재고 차감 + 주문 생성(ORDERS/ORDER_ITEMS INSERT)을 같은 트랜잭션에서 처리하고, 트랜잭션 확정 후 장바구니를 비운 뒤 redirect:/order/{orderNo} (기존 주문 상세 화면)로 이동한다. 재조회하면 장바구니는 빈 상태다.
4. 실패(400 빈 장바구니 · 404 회원 없음 · 409 재고부족/판매중지/상품없음 — 사전 스윙 또는 OrderService.create의 실제 차감 실패로 판정, 전량 거부)면 컨트롤러가 **4xx만 흡수**해 redirect:/cart?memberId=... + flash checkoutError로 돌아온다. 화면 상단에 "체크아웃 실패: {사유}" 문구가 노출되고(list.html:28-30) 장바구니는 그대로 보존된다(트랜잭션 롤백). 409 사유는 품목별로 모아 "재고 부족: SKU-1001(가용1/요청2), SKU-1002(판매중지)" 형태로 한 번에 전달된다(D9 — 재고부족+판매중지+미존재 상품을 포괄하는 단일 계열, 접두어는 현행 "재고 부족:"로 통일 전). 5xx는 흡수하지 않고 그대로 전파된다(장애 은폐 금지).

3. 화면 구성 (블록)

블록	역할	주요 위젯	소스 근거
회원 선택 영역	조회 대상(장바구니 소유자) 전환	(1) 회원 셀렉트, (2) 조회 버튼	list.html: 33-42
품목 표 (품목 있음)	상품명·단가·수량·품목합계 표시 + 행별 수량변경/삭제	(3) 수량 input, (4) 변경, (5) 삭제 (행마다 반복)	list.html: 44-70
총합계	전 품목 합계 표시	정적 텍스트 (cart.totalAmount)	list.html: 71-73
주문하기 영역 (품목 있음)	장바구니 전체를 주문으로 전환(체크아웃)	(6) 주문하기 버튼/폼	list.html: 75-80
빈 상태 안내 (품목 없음)	"장바구니가 비어 있습니다" + 이동 링크	정적 텍스트 + a	list.html: 82-86
하단 네비게이션	상품 목록으로 복귀(품목 있을 때도 항상 노출)	(7) a 링크	list.html: 88
체크아웃 실패 안내 (품목 있음, 실패 후 재진입)	직전 체크아웃 실패 사유 표시	정적 텍스트(checkoutError flash)	list.html: 28-30

4. 위젯·액션

번호는 「0. 화면 미리보기」에서 설명한 대로 **DOM 스냅샷 위젯 번호(n)** 를 그대로 쓴다 (annotate 마커
아님 — id/onclick 보유 위젯이 3/7뿐이라 자동 선택은 전수를 만족 못 함).

번호	위젯	타입	레이블	동작	연결 API(raw)	결과
(1)	#selMember	select	회원 (이름 (ID) 목록)	회원 전환 시 onchange로 자동 제출	GET /cart	선택 회원의 장바구니로 재조회
(2)	#btnSelect	button(submit)	조회	회원 선택 품수동 재제출(백업 수단)	GET /cart	그리드 갱신(현재 선택 회원 기준)
(3)	qty input (행별)	input(number, min=1)	수량	값 수정	— (같은 행 (4)의 파라미터로 제출)	—
(4)	변경 버튼 (행별)	button(submit)	변경	수량변경 품 제출(hidden memberId + qty)	POST /cart/items/{sku}/update	redirect:/cart → 수량·품목합계·총합계 갱신. 실패: 400(qty<1)/409(재고초과)
(5)	삭제 버튼 (행별)	button(submit)	삭제	명시적 삭제(D4 — 수량 0 유도 금지)	POST /cart/items/{sku}/delete	redirect:/cart → 해당 행 제거, 총합계 갱신
(6)	#btnCheckout	button(submit)	주문하기	체크아웃 품 제출(hidden memberId)	POST /cart/checkout	성공: redirect:/order/{orderNo}(장바구니 비워짐). 실패(400/404/409): redirect:/cart+flash checkoutError, 장바구니 보존
(7)	a[href=/product/list]	a	상품 목록으로	화면 이동	GET /product/list	상품 목록(UIS-ORD-003)으로 이동, 담기 재진입

5. 접근 권한·표시 조건

요소	표시 조건	근거
회원 선택 옵션	MemberService.list() 전체 회원(권한 분기 없음)	list.html:36-38, CartViewController.java:42
품목 표 + 총합계	cart.items가 비어있지 않을 때만 렌더	list.html:44,71
[주문하기] 버튼/품(6)	cart.items가 비어있지 않을 때만 렌더(D6 — 빈 장바구니는 동작하지 않는다)	list.html:77
빈 상태 안내("장바구니가 비어 있습니다")	cart.items가 비어있을 때만 렌더	list.html:83

요소	표시 조건	근거
하단 "상품 목록으로" 링크 ((7))	cart.items가 비어있지 않을 때 별도로 다시 노출 (빈 상태 링크와 별개 위치)	list.html:88
각 행의 [변경]/[삭제] 품	품목이 존재하는 행마다 항상 노출(권한·상태 분기 없음)	list.html:52-66
"체크아웃 실패: {사유}" 안내	직전 요청의 checkoutError 플래시 속성이 있을 때만 렌더(1회성)	list.html:28-30

6. 팝업·연계 화면

팝업 없음. 이동 대상만 기록.

트리거 위젯	팝업/연계 화면	연결 API/화면 (raw → INF)	용도
(7) a[href=/product/list] (빈 상태·품목 있음 상태 공통)	상품 목록 화면(UIS-ORD-003)	GET /product/list ← 화면 이동(INF 없음)	목록에서 추가 상품 담기(UIS-ORD-004로 이어짐)
(6) #btnCheckout (성공 시)	주문 상세 화면(기존, /order/{orderNo})	POST /cart/checkout ← 화면 이동(INF 없음, 대응 REST는 [[INF-ORD-014]])	체크아웃 성공 후 생성된 주문 확인

7. 데이터 출처·연결

- **연결 API(raw → INF):**
- GET /cart — 화면 자체 라우트(서버 렌더 진입점). INF 대상 아님(view 반환). 내부적으로 CartService.get(memberId)를 자바 메서드로 직접 호출한다(HTTP 아님).
- POST /cart/items/{sku}/update, POST /cart/items/{sku}/delete — 서버렌더 품 제출(PRG, redirect 반환). JSON API가 아니라 INF 대상 아님. 내부적으로 CartService.updateQty/CartService.delete를 자바 메서드로 직접 호출하며, 이는 **REST API 컨트롤러(CartController)와 동일 서비스 인스턴스를 공유한다** — 다만 이 화면은 그 REST API를 HTTP로 부르지 않는다. 같은 서비스가 노출하는 대응 REST
엔드포인트: 담기 [\[INF-ORD-010\]](#), 조회 [\[INF-ORD-011\]](#), 수량변경 [\[INF-ORD-012\]](#), 삭제 [\[INF-ORD-013\]](#).
- POST /cart/checkout — 서버렌더 품 제출(PRG, redirect 반환). JSON API가 아니라 INF 대상 아님. 내부적으로 CartService.checkout(memberId)를 자바 메서드로 직접 호출하며, OrderService.create를 그대로 재사용해 재고 차감·주문 생성을 같은 트랜잭션으로 처리한다(SR-203 D6-D7, 새 주문 규칙 복제 금지). 같은 비즈니스 로직을 노출하는 대응 REST 엔드포인트: 체크아웃 [\[\[INF-ORD-014\]\]](#)

(POST /api/cart/checkout) — SR-203에서 신설, 409 응답에 품목별 사유 목록을 담아 전량 거부한다(D9).

- GET /product/list — 목록 화면 이동. INF 대상 아님.
- **참조 테이블(SCH):** CART_ITEMS, PRODUCTS, MEMBERS, ORDERS, ORDER_ITEMS(INF-ORD-010~014 tables 기준) — 단 **CART_ITEMS의 SCH 문서는 아직 생성되지 않았다**(docs/05_설계서/order/SCH/에 SCH-ORD-001~006만 존재, CART_ITEMS 언급 0건) — /sl-recon-sch 재실행 대상. PRODUCTS는 [[SCH-ORD-005]].

8. 미확인 사항

- **REST API(/api/cart*)와 화면 폼(/cart/*)은 별개 경로다.** 이 화면의 수량변경·삭제·체크아웃은 브라우저가 직접 REST API(PATCH/DELETE /api/cart/items/{sku}, POST /api/cart/checkout)를 호출하는 게 아니라, 서버렌더 폼이 CartViewController의 자체 라우트로 POST하고 그 안에서 CartService를 자바 메서드로 호출한다. REST API(담기 [[INF-ORD-010]], 조회 [[INF-ORD-011]], 수량변경 [[INF-ORD-012]], 삭제 [[INF-ORD-013]], 체크아웃 [[INF-ORD-014]])는 별도 클라이언트(외부 연동 등)를 위한 것으로 보이며 같은 비즈니스 로직을 공유할 뿐 이 화면이 호출하는 경로가 아니다 — 두 계층을 혼동하지 않도록 api_hints도 화면 경로(/cart/*) 위주로 담았고, 체크아웃 REST 대응만 SR-203 지시에 따라 예외적으로 POST /api/cart/checkout을 api_hints에 병기했다(INF-ORD-014 역참조 연결용).
- **폼 오류 시 Whitelabel 이탈(QA r1~r3 CONCERNS, 잔존 이월 — 수량변경/삭제 한정):** qty<1(400) 또는 재고 초과(409)로 수량변경이 실패하면 이 화면으로 재렌더되지 않고 Spring 기본 오류 페이지(Whitelabel)로 이동한다. 사유 문구 자체는 응답 본문에 정상 전달되지만(server.error.include-message: always), 장바구니 화면 컨텍스트는 유지되지 않는다. **체크아웃(SR-203)은 이 문제를 겪지 않는다** — CartViewController#checkout이 4xx를 명시적으로 흡수해 redirect:/cart + flash로 화면 컨텍스트를 보존하도록 새로 구현됐다. QA에서 권고로 남기고 수용됨(STORY-FUNC-order-011.md 후속 추적 TODO) — 수량변경/삭제까지 화면 재렌더+사유 표시로 확장하는 것은 이번 범위 밖.
- **CSRF 미방어(D3/QA 결정 수용):** /cart/items/{sku}/update·/cart/items/{sku}/delete·/cart/checkout은 앱 전역에 spring-security가 없어 CSRF 토큰이 없다. 사람 코멘트로 이번 SR 범위에서 명시 수용됐고 후속 SR/KNOWN_LIMITATIONS 등재가 후속 추적 TODO로 남아 있다(STORY-FUNC-order-011.md 참조).

- **수량 1 미만은 거부만, 삭제 유도 없음(D4):** qty<1 입력 시 자동으로 삭제 처리되지 않고 400으로 거부된다 — 사용자가 품목을 없애려면 반드시 (5) [삭제] 버튼을 눌러야 한다. 소스 주석 (list.html:57 대응 원칙)에도 이 원칙이 명시돼 있다.
- **CART_ITEMS SCH 미생성:** 7번 참조 — 이번 UIS 재작성 시점에도 아직 없어 테이블명만 기록했다. /sl-recon-sch 재실행 대상.
- **409 메시지 접두어 통일 미완(D9):** 체크아웃 409는 재고부족·판매중지·미존재 상품을 포괄하는 단일 사유 계열로 확정됐으나, 현재 구현은 세 사유 모두 "재고 부족:" 접두어를 그대로 쓴다(예: 판매중지 품목도 "재고 부족: SKU-x(판매중지)"로 표기). 메시지 접두어 자체를 통일하는 것은 후속 추적(비차단, D9).

API 설계서

INF-ORD-001

INF-ORD-001: GET /api/members/ — 회원 목록 조회

개요: 탈퇴하지 않은 회원 전체를 회원ID 순으로 조회한다.

근거 소스: src/main/java/com/sm/lab/order/controller/MemberController.java:21-25

요청

- Method: GET
- Path: /api/members/
- Content-Type: application/json

파라미터	위치	타입	필수	설명
(없음)	-	-	-	-

응답 (200 OK)

```
[
  {
    "memberId": "string",
    "memberName": "string",
    "grade": "BRONZE | SILVER | GOLD | VIP",
    "phone": "string",
    "createdAt": "2026-01-01T00:00:00",
    "recentOrders": []
  }
]
```

비즈니스 규칙

- del_yn = 'N' 상시필터 → 탈퇴 회원은 목록에서 제외
- recentOrders는 이 API에서 채워지지 않고 항상 빈 배열([INF-ORD-002])에서만 채워짐

오류 응답

코드	사유	발생 조건
400	유효성 실패	필수 필드 누락

코드	사유	발생 조건
401	인증 실패	토큰 없음/만료

참조 테이블

테이블	SCH
MEMBERS	[[SCH-ORD-001]]

curl 예시

```
curl -X GET /api/members/ \
  -H "Content-Type: application/json"
```

INF-ORD-002 회원 상세 조회

INF-ORD-002: GET /api/members/{memberId} — 회원 상세 조회

개요: 회원 상세 정보와 함께 최근 주문 5건 요약을 반환한다. (linked_func: FUNC-order-004, LAB-103)

근거 소스: modules/order-

api/src/main/java/com/sm/lab/order/controller/MemberController.java:27-31

요청

- Method: GET
- Path: /api/members/{memberId}
- Content-Type: application/json

파라미터	위치	타입	필수	설명
memberId	PathVariable	string	Y	회원 ID

응답 (200 OK)

```
{
  "memberId": "string",
  "memberName": "string",
  "grade": "BRONZE | SILVER | GOLD | VIP",
  "phone": "string",
  "createdAt": "2026-01-01T00:00:00",
  "recentOrders": [
    {
      "orderNo": "string",
      "orderState": "PLACED | PAID | SHIPPED | PARTIAL_SHIPPED | CANCELED | DONE",
      "totalAmount": 0,
      "orderedAt": "2026-01-01T00:00:00"
    }
  ]
}
```

비즈니스 규칙

- del_yn = 'N' 상시필터 → 탈퇴 회원 조회 시 404
- recentOrders는 최근 주문 상위 5건 고정(RECENT_ORDER_LIMIT), ordered_at DESC, order_no DESC 순

- 주문이 없으면 recentOrders는 빈 배열(null 금지, Member 도메인 필드 기본값)

오류 응답

코드	사유	발생 조건
404	회원 없음	memberId로 조회된 회원이 없거나 탈퇴 회원

참조 테이블

- MEMBERS
- ORDERS

curl 예시

```
curl -X GET /api/members/M0001 \  
-H "Content-Type: application/json"
```

INF-ORD-003

INF-ORD-003: GET /api/orders/ — 주문 목록 조회

개요: 회원·주문상태·조회기간으로 필터링한 주문 목록을 페이징 조회한다. (linked_func: FUNC-order-001, LAB-101)

[반영: FUNC-order-001] SR-205 — 조회 기간(startDate/endDate) 필터 추가.

근거 소스: src/main/java/com/sm/lab/order/controller/OrderController.java:28-44

요청

- Method: GET
- Path: /api/orders/
- Content-Type: application/json

파라미터	위치	타입	필수	설명
memberId	Query	string	N	회원 ID 필터
orderState	Query	string	N	주문상태 필터
startDate	Query	date(ISO-8601)	N	조회 시작일. 미제시 시 종료일 기준 30일 전(오늘-30일)으로 독립 보장. [반영: FUNC-order-001]
endDate	Query	date(ISO-8601)	N	조회 종료일. 미제시 시 오늘로 독립 보장. startDate/endDate는 각각 독립적으로 기본값이 적용된다(하나만 제시해도 나머지만 보장). [반영: FUNC-order-001]
page	Query	int	N	페이지 번호(기본 1)
size	Query	int	N	페이지 크기(기본 20)

응답 (200 OK)

```
{
  "totalCount": 0,
  "page": 1,
  "items": [
    {
      "orderNo": "string",
      "memberId": "string",
      "memberName": "string",
```

```

    "orderState": "PLACED | PAID | SHIPPED | PARTIAL_SHIPPED | CANCELED | DONE",
    "totalAmount": 0,
    "orderedAt": "2026-01-01T00:00:00",
    "items": null,
    "deliveries": null
  }
]
}

```

비즈니스 규칙

- del_yn = 'N' 상시 필터 → 논리삭제된 주문 제외
- memberId 파라미터가 있으면 해당 회원 주문만, orderState 파라미터가 있으면 해당 상태만 필터 (LAB-101 추가 요구사항)
- offset = max(0, page - 1) * size
- ORDERS와 MEMBERS를 조인해 memberName을 함께 반환(회원 미탈퇴 여부와 무관하게 조인만 수행)
- 목록 응답의 각 항목은 items/deliveries를 채우지 않음(상세는 [[INF-ORD-004]])
- **[반영: FUNC-order-001]** 기간 필터는 orderState/memberId와 **AND**로 결합되며, ordered_at(DATETIME) 기준
양끝 날짜를 포함한다(ordered_at >= startDate AND ordered_at < endDate + 1일). 응답 스키마(totalCount/page/items[])는 변경하지 않는다 — 요청 측 파라미터만 추가.
- **[반영: FUNC-order-001]** startDate/endDate는 각각 독립적으로 기본값이 적용된 뒤 유효 구간(effectiveStart~effectiveEnd)이
역전되면(예: endDate만 제시해 보정된 effectiveStart가 그보다 늦어지는 경우) 조회를 수행하지 않고 400으로 거부한다.

오류 응답

코드	사유	발생 조건
400	유효성 실패	파라미터 타입 오류(날짜 형식 포함) 또는 유효 조회기간 역전(effectiveStart > effectiveEnd) [반영: FUNC-order-001]
401	인증 실패	토큰 없음/만료

인증·인가 (SR-204 — 전역 정책)

이 오류 계약은 /api/** 전역에 적용되는 횡단 정책이다(구현: ApiKeyAuthFilter). 화면 라우트(/product/./cart/order/)는 무인증 유지. 개별 엔드포인트 INF는 이 정책을 상속한다.

조건	상태	본문
x-api-key 미제시/무효	401	{"error": "unauthorized"}

조건	상태	본문
member 스코프 키로 타 회원 자원 접근(소유권 불일치)	403	{"error": "forbidden"}
회원 목록·주문 export 등 관리 자원에 member 키	403	admin 키 전용

- 키 매핑은 앱 설정 lab.api-keys(admin→*, member→회원ID). 경로 판정은 서블릿 정규화 경로 기준(퍼센트 인코딩·matrix 파라미터 우회 차단, default-deny). 소유권은 자원의 실제 member_id를 DB 대조(orders/{orderNo}) — 토큰 문자열 대조가 아니다(SR-204 D15).

참조 테이블

테이블	SCH
ORDERS	[[SCH-ORD-004]]
MEMBERS	[[SCH-ORD-001]]

curl 예시

```
curl -X GET "/api/orders/?memberId=M0001&orderState=PLACED&page=1&size=20" \
-H "Content-Type: application/json"
```

INF-ORD-004

INF-ORD-004: GET /api/orders/{orderNo} — 주문 상세 조회

개요: 주문 1건을 라인(아이템)·배송 정보를 포함해 조회한다.

근거 소스: src/main/java/com/sm/lab/order/controller/OrderController.java:31-34

요청

- Method: GET
- Path: /api/orders/{orderNo}
- Content-Type: application/json

파라미터	위치	타입	필수	설명
orderNo	PathVariable	string	Y	주문번호

응답 (200 OK)

```
{
  "orderNo": "string",
  "memberId": "string",
  "memberName": "string",
  "orderState": "PLACED | PAID | SHIPPED | PARTIAL_SHIPPED | CANCELED | DONE",
  "totalAmount": 0,
  "orderedAt": "2026-01-01T00:00:00",
  "items": [
    {
      "orderNo": "string",
      "lineNo": 1,
      "sku": "string",
      "productName": "string",
      "qty": 0,
      "unitPrice": 0
    }
  ],
  "deliveries": [
    {
      "deliveryNo": "string",
      "orderNo": "string",
      "deliveryState": "READY | SHIPPED | DELIVERED | CANCELED",
      "invoiceNo": "string",
      "shippedAt": "2026-01-01T00:00:00"
    }
  ]
}
```

```
]
}
```

비즈니스 규칙

- del_yn = 'N' 상시필터 → 논리삭제된 주문 조회 시 404
- items는 ORDER_ITEMS와 PRODUCTS를 조인해 productName을 함께 반환
- deliveries는 배송이 없으면 빈 배열

오류 응답

코드	사유	발생 조건
404	주문 없음	orderNo로 조회된 주문이 없거나 삭제됨

참조 테이블

테이블	SCH
ORDERS	[[SCH-ORD-004]]
MEMBERS	[[SCH-ORD-001]]
ORDER_ITEMS	[[SCH-ORD-003]]
PRODUCTS	[[SCH-ORD-005]]
ORDER_DELIVERY	[[SCH-ORD-002]]

curl 예시

```
curl -X GET /api/orders/20260101-0101 \
  -H "Content-Type: application/json"
```

INF-ORD-005

INF-ORD-005: POST /api/orders/ — 주문 생성

개요: 요청된 상품 라인들을 검증·재고 차감한 뒤 신규 주문을 생성한다.

근거 소스: src/main/java/com/sm/lab/order/controller/OrderController.java:36-40

요청

- Method: POST
- Path: /api/orders/
- Content-Type: application/json

```
{
  "memberId": "string",
  "items": [
    { "sku": "string", "qty": 0 }
  ]
}
```

파라미터	위치	타입	필수	설명
memberId	Body	string	Y	회원 ID
items	Body	array	Y	주문 라인 목록(sku, qty)

응답 (200 OK)

```
{
  "orderNo": "string",
  "memberId": "string",
  "memberName": "string",
  "orderState": "PLACED",
  "totalAmount": 0,
  "orderedAt": "2026-01-01T00:00:00",
  "items": [ { "orderNo": "string", "lineNo": 1, "sku": "string", "productName": "string", "qty": 0,
    "unitPrice": 0 } ],
  "deliveries": []
}
```

비즈니스 규칙

- 각 라인의 상품이 sale_yn = 'Y'(판매중)가 아니면 400

- 재고 차감(decreaseStock)이 0행이면 재고 부족으로 409 — WHERE stock_qty >= qty 조건부 UPDATE로 동시성 가드
- 주문번호(orderNo)는 yyyyMMdd-{4자리 시퀀스} 형식으로 채번(AtomicInteger, 시작값 100 — 인스턴스 내 메모리 채번, 재기동 시 초기화)
- 신규 주문 상태는 항상 PLACED
- totalAmount는 각 라인 단가 × 수량의 합

트랜잭션 순서

1. 라인별로 PRODUCTS 조회 → 판매상태 검증 → 재고 조건부 차감(UPDATE ... WHERE stock_qty >= qty)
2. ORDERS INSERT (order_state='PLACED')
3. 라인별 ORDER_ITEMS INSERT
4. 생성된 주문을 [[INF-ORD-004]]와 동일한 조회로 재조회하여 반환

오류 응답

코드	사유	발생 조건
400	판매 중인 상품이 아님	상품 미존재 또는 sale_yn != 'Y'
409	재고 부족	차감 대상 수량이 현재 재고보다 많음

참조 테이블

테이블	SCH
PRODUCTS	[[SCH-ORD-005]]
ORDERS	[[SCH-ORD-004]]
ORDER_ITEMS	[[SCH-ORD-003]]
MEMBERS	[[SCH-ORD-001]]
ORDER_DELIVERY	[[SCH-ORD-002]]

curl 예시

```
curl -X POST /api/orders/ \
-H "Content-Type: application/json" \
-d '{"memberId":"M0001","items":[{"sku":"SKU001","qty":2}]}'
```

INF-ORD-006

INF-ORD-006: PATCH

/api/orders/{orderNo}/cancel — 주문 취소

개요: 출고 전 주문을 취소하고 차감했던 재고를 원복한다. (linked_func: FUNC-order-002, LAB-102)

근거 소스: src/main/java/com/sm/lab/order/controller/OrderController.java:42-46

요청

- Method: PATCH
- Path: /api/orders/{orderNo}/cancel
- Content-Type: application/json

파라미터	위치	타입	필수	설명
orderNo	PathVariable	string	Y	주문번호

응답 (200 OK)

```
{
  "orderNo": "string",
  "memberId": "string",
  "memberName": "string",
  "orderState": "CANCELED",
  "totalAmount": 0,
  "orderedAt": "2026-01-01T00:00:00",
  "items": [ { "orderNo": "string", "lineNo": 1, "sku": "string", "productName": "string", "qty": 0,
    "unitPrice": 0 } ],
  "deliveries": []
}
```

비즈니스 규칙

- 현재 상태가 CANCELED 또는 DONE이면 409(이미 종결된 주문)
- 배송 중 하나라도 SHIPPED 또는 DELIVERED 상태면 409 ORD-4001(출고 완료 배송은 취소 불가)
- 상태 전이를 **먼저** 조건부 UPDATE(WHERE order_state NOT IN ('CANCELED', 'DONE'))로 확정한다

—
0행이면 동시 요청이 선점한 것으로 보고 재고 원복 없이 즉시 409(레이스 가드, LAB-102 QA r1)

- 재고 원복은 상태 전이가 확정된 뒤 라인별로 수행

트랜잭션 순서

1. 주문 상세 재조회(상태·배송상태 확인)
2. ORDERS.order_state 조건부 UPDATE → CANCELED(레이스การ์ด)
3. 각 라인(ORDER_ITEMS)의 수량만큼 PRODUCTS.stock_qty 증가(원복)
4. 갱신된 주문 상세 재조회 후 반환

오류 응답

코드	사유	발생 조건
404	주문 없음	orderNo로 조회된 주문이 없음
409	취소 불가 상태	이미 CANCELED/DONE
409	ORD-4001 출고 완료 배송은 취소할 수 없습니다	배송상태가 SHIPPED/DELIVERED인 배송 존재
409	취소 불가 상태(동시 처리됨)	상태 전이 UPDATE가 0행(레이스)

참조 테이블

테이블	SCH
ORDERS	[[SCH-ORD-004]]
MEMBERS	[[SCH-ORD-001]]
ORDER_ITEMS	[[SCH-ORD-003]]
PRODUCTS	[[SCH-ORD-005]]
ORDER_DELIVERY	[[SCH-ORD-002]]

curl 예시

```
curl -X PATCH /api/orders/20260101-0101/cancel \
  -H "Content-Type: application/json"
```

INF-ORD-007

INF-ORD-007: GET

/api/orders/{orderNo}/deliveries — 주문 배송 목록 조회

개요: 주문 1건의 배송 목록만 반환한다.

근거 소스: src/main/java/com/sm/lab/order/controller/OrderController.java:48-52

요청

- Method: GET
- Path: /api/orders/{orderNo}/deliveries
- Content-Type: application/json

파라미터	위치	타입	필수	설명
orderNo	PathVariable	string	Y	주문번호

응답 (200 OK)

```
[
  {
    "deliveryNo": "string",
    "orderNo": "string",
    "deliveryState": "READY | SHIPPED | DELIVERED | CANCELED",
    "invoiceNo": "string",
    "shippedAt": "2026-01-01T00:00:00"
  }
]
```

비즈니스 규칙

- 내부적으로 [[INF-ORD-004]]와 동일한 OrderService.detail()을 호출한다 — ORDER_ITEMS도 함께 조회되지만
응답에는 deliveries만 반환된다(불필요한 내부 조회이나 기능상 영향 없음)
- 배송이 없으면 빈 배열

오류 응답

코드	사유	발생 조건
404	주문 없음	orderNo로 조회된 주문이 없음

참조 테이블

테이블	SCH
ORDERS	[[SCH-ORD-004]]
MEMBERS	[[SCH-ORD-001]]
ORDER_ITEMS	[[SCH-ORD-003]]
ORDER_DELIVERY	[[SCH-ORD-002]]

curl 예시

```
curl -X GET /api/orders/20260101-0101/deliveries \
-H "Content-Type: application/json"
```

변경 이력

날짜	변경 내용	변경자
2026-08-23	배송 이력 정렬을 최신순으로(shipped_at DESC, 미발송 null은 하단 · tie는 delivery_no DESC) — LAB-113, linked_func: FUNC-order-006	/sl-change --quick (auto)

INF-ORD-008

[변경: SR-201] 2026-08-22

INF-ORD-008: GET /api/products — 판매중 상품 목록 조회

개요: 판매중 상태인 상품 전체를 SKU 순으로 조회한다.

근거 소스: src/main/java/com/sm/lab/order/controller/ProductController.java:24-31

요청

- Method: GET
- Path: /api/products
- Content-Type: application/json

파라미터	위치	타입	필수	설명
keyword	Query	string	N	상품명 부분 일치 검색어. 미지정 시 전체(판매중) 목록 반환(하위 호환) — [반영: FUNC-order-007] 2026-08-22

응답 (200 OK)

```
[
  {
    "sku": "string",
    "productName": "string",
    "price": 0,
    "stockQty": 0,
    "saleYn": "Y"
  }
]
```

비즈니스 규칙

- sale_yn = 'Y' 상시필터 → 판매중 상품만 반환(단건 조회인 [[INF-ORD-009]]는 이 필터가 없음)
- [반영: FUNC-order-007] keyword 파라미터가 있으면 product_name 부분 일치(LIKE) 검색 — sale_yn = 'Y' 상시필터와 AND 결합. keyword 미지정 시 기존과 동일하게 전체(판매중) 목록 반환(하위 호환),

기존 소비자 무영향). 응답 스키마(sku/productName/price/stockQty/saleYn) 변경 없음 —
"품질" 판정은 화면(UIS-ORD-003) 몫으로 stockQty 값을 그대로 사용.

오류 응답

코드	사유	발생 조건
400	유효성 실패	필수 필드 누락
401	인증 실패	토큰 없음/만료

참조 테이블

테이블	SCH
PRODUCTS	[[SCH-ORD-005]]

curl 예시

```
curl -X GET /api/products \
  -H "Content-Type: application/json"
```

[반영: FUNC-order-007] keyword 검색 예시:

```
curl -X GET "/api/products?keyword=셔츠" \
  -H "Content-Type: application/json"
```

변경 이력

날짜	변경 내용	변경자
2026-08-22	SR-201: keyword 부분일치 검색 파라미터 추가(구현 반영 — FUNC-order-007). path 표기를 실측 라우트(/api/products — 트레일링 슬래시 404)로 정정	sl-aidd STEP 5.5 재동기화

INF-ORD-009

INF-ORD-009: GET /api/products/{sku} — 상품 단건 조회

개요: SKU로 상품 1건을 조회한다.

근거 소스: src/main/java/com/sm/lab/order/controller/ProductController.java:27-31

요청

- Method: GET
- Path: /api/products/{sku}
- Content-Type: application/json

파라미터	위치	타입	필수	설명
sku	PathVariable	string	Y	상품 SKU

응답 (200 OK)

```
{
  "sku": "string",
  "productName": "string",
  "price": 0,
  "stockQty": 0,
  "saleYn": "Y"
}
```

비즈니스 규칙

- 목록 조회([INF-ORD-008])와 달리 sale_yn 필터가 없다 — 판매종료 상품도 SKU를 알면 조회 가능

오류 응답

코드	사유	발생 조건
404	상품 없음	sku로 조회된 상품이 없음

참조 테이블

테이블	SCH
PRODUCTS	[[SCH-ORD-005]]

curl 예시

```
curl -X GET /api/products/SKU001 \  
-H "Content-Type: application/json"
```

INF-ORD-010

INF-ORD-010: POST /api/cart/items — 장바구니 담기

개요: 회원 장바구니에 상품을 담는다. 같은 상품을 재담기하면 기존 수량에 합산된다.

근거 소스: src/main/java/com/sm/lab/order/controller/CartController.java:22-26

요청

- Method: POST
- Path: /api/cart/items
- Content-Type: application/json

파라미터	위치	타입	필수	설명
memberId	Body	string	Y	회원 ID
sku	Body	string	Y	상품 SKU
qty	Body	int	Y	담을 수량 (1 이상)

응답 (200 OK)

```
{
  "memberId": "string",
  "sku": "string",
  "productName": "string",
  "price": 0,
  "qty": 0,
  "addedAt": "2026-08-23T10:00:00",
  "lineTotal": 0
}
```

비즈니스 규칙

- 같은 상품 재담기는 DB 원자 UPSERT(INSERT ... ON DUPLICATE KEY UPDATE qty = qty + VALUES(qty))로 수량 합산(PK member_id, sku) — 동시 담기에도 lost update-PK 중복 500 없음
- 판매중지(sale_yn='N') 상품은 담기 거부(409)
- 품절(stock_qty=0)은 UPSERT 이전에 조기 거부(409) — 합산·원복 사이클 자체가 불필요
- UPSERT 직후 최종(합산) qty가 재고를 초과하면, 방금 더한 만큼만 되돌리고(원복값 0 이하면 행 삭제) 409 — UPSERT가 잡은 행 잠금이 트랜잭션 종료까지 유지되어 판정·원복 사이 다른 트랜잭션 개

입 불가

- 담기는 재고를 차감하지 않는다(보관 전용)

트랜잭션 순서

1. CART_ITEMS UPSERT(수량 합산)
2. CART_ITEMS 병합 결과 SELECT(상품명·현재 단가 조인)
3. (재고 초과 시) CART_ITEMS 원복 — 복원값 > 0이면 UPDATE, 0 이하면 DELETE

오류 응답

코드	사유	발생 조건
400	유효성 실패	qty < 1
404	회원 없음	memberId로 조회된 회원 없음
404	상품 없음	sku로 조회된 상품 없음
409	판매중지 상품	상품의 sale_yn = 'N'
409	품절 상품	상품의 stock_qty = 0
409	재고 초과	합산된 qty가 가용 재고 초과

참조 테이블

테이블	SCH
CART_ITEMS	[[SCH-ORD-006]]
PRODUCTS	[[SCH-ORD-005]]
MEMBERS	[[SCH-ORD-001]]

curl 예시

```
curl -X POST /api/cart/items \
  -H "Content-Type: application/json" \
  -d '{"memberId":"M001","sku":"SKU001","qty":2}'
```

INF-ORD-011

INF-ORD-011: GET /api/cart/ — 장바구니 조회

개요: 회원의 장바구니 품목 전체와 총액을 조회한다.

근거 소스: src/main/java/com/sm/lab/order/controller/CartController.java:28-32

요청

- Method: GET
- Path: /api/cart/
- Content-Type: application/json

파라미터	위치	타입	필수	설명
memberId	Query	string	Y	회원 ID

응답 (200 OK)

```
{
  "items": [
    {
      "memberId": "string",
      "sku": "string",
      "productName": "string",
      "price": 0,
      "qty": 0,
      "addedAt": "2026-08-23T10:00:00",
      "lineTotal": 0
    }
  ],
  "totalAmount": 0
}
```

비즈니스 규칙

- 품목은 담은 순(added_at, sku)으로 정렬된다
- totalAmount는 각 품목의 price × qty(lineTotal) 합계로, 조회 시점에 애플리케이션에서 계산한다 (저장 컬럼 아님)

오류 응답

코드	사유	발생 조건
404	회원 없음	memberId로 조회된 회원 없음

참조 테이블

테이블	SCH
CART_ITEMS	[[SCH-ORD-006]]
PRODUCTS	[[SCH-ORD-005]]
MEMBERS	[[SCH-ORD-001]]

curl 예시

```
curl -X GET "/api/cart/?memberId=M001" \
-H "Content-Type: application/json"
```

INF-ORD-012

INF-ORD-012: PATCH /api/cart/items/{sku} — 장바구니 수량 변경

개요: 장바구니에 담긴 품목의 수량을 변경한다.

근거 소스: src/main/java/com/sm/lab/order/controller/CartController.java:34-39

요청

- Method: PATCH
- Path: /api/cart/items/{sku}
- Content-Type: application/json

파라미터	위치	타입	필수	설명
sku	PathVariable	string	Y	상품 SKU
memberId	Query	string	Y	회원 ID
qty	Body	int	Y	변경할 수량 (1 이상)

응답 (200 OK)

```
{
  "memberId": "string",
  "sku": "string",
  "productName": "string",
  "price": 0,
  "qty": 0,
  "addedAt": "2026-08-23T10:00:00",
  "lineTotal": 0
}
```

비즈니스 규칙

- qty < 1은 400으로 거부한다 — 삭제는 명시적 DELETE로만 수행(실수 삭제 방지)
- 요청 qty가 상품 재고를 초과하거나 상품이 품절(stock_qty=0)이면 409

트랜잭션 순서

1. CART_ITEMS UPDATE(qty 변경)
2. CART_ITEMS SELECT(변경 결과 반환용, 상품명·현재 단가 조인)

오류 응답

코드	사유	발생 조건
400	유효성 실패	qty < 1
404	장바구니 품목 없음	memberId+sku 조합의 품목 없음
404	상품 없음	sku로 조회된 상품 없음
409	재고 초과/품절	요청 qty가 가용 재고 초과 또는 상품 품절

참조 테이블

테이블	SCH
CART_ITEMS	[[SCH-ORD-006]]
PRODUCTS	[[SCH-ORD-005]]

curl 예시

```
curl -X PATCH "/api/cart/items/SKU001?memberId=M001" \
  -H "Content-Type: application/json" \
  -d '{"qty":3}'
```

INF-ORD-013

INF-ORD-013: DELETE /api/cart/items/{sku} — 장바구니 품목 삭제

개요: 장바구니에서 품목 1건을 삭제한다.

근거 소스: src/main/java/com/sm/lab/order/controller/CartController.java:41-46

요청

- Method: DELETE
- Path: /api/cart/items/{sku}
- Content-Type: application/json

파라미터	위치	타입	필수	설명
sku	PathVariable	string	Y	상품 SKU
memberId	Query	string	Y	회원 ID

응답 (204 No Content)

응답 본문 없음.

오류 응답

코드	사유	발생 조건
404	장바구니 품목 없음	memberId+sku 조합의 품목 없음

참조 테이블

테이블	SCH
CART_ITEMS	[[SCH-ORD-006]]
PRODUCTS	[[SCH-ORD-005]]

curl 예시

```
curl -X DELETE "/api/cart/items/SKU001?memberId=M001" \
-H "Content-Type: application/json"
```

INF-ORD-014

INF-ORD-014: POST /api/cart/checkout — 장바구니 체크아웃

개요: 장바구니 전체 품목을 기존 주문 생성 규칙으로 전환해 주문을 만들고 장바구니를 비운다.

근거 소스: src/main/java/com/sm/lab/order/controller/CartController.java:53-56

요청

- Method: POST
- Path: /api/cart/checkout
- Content-Type: application/json

파라미터	위치	타입	필수	설명
memberId	Body	string	Y	회원 ID

```
{
  "memberId": "M001"
}
```

응답 (200 OK)

```
{
  "orderNo": "20260823-0101",
  "totalAmount": 0,
  "itemCount": 0
}
```

비즈니스 규칙

- 주문 생성은 새 규칙을 만들지 않고 `OrderService.create`를 그대로 재사용한다(채번·PLACED 상태·재고 차감 — 검증/차감 로직 미복제)
- 장바구니 조회는 `SELECT ... FOR UPDATE`로 잠근다 — 동일 회원의 동시 체크아웃에서 스냅샷이 아닌 최신 커밋 데이터를 잠그므로, 중복 주문이 발생하지 않는다
- 잠금 직후 전 품목을 사전 스왑하여 상품없음·판매중지(`sale_yn='N'`)·재고부족을 한 번에 모아 409 사유 목록 하나로 응답한다(품목별 사유, 부분 주문 없이 전량 거부)
- 사전 스왑을 통과해도 `OrderService.create`의 실제 재고 차감(`decreaseStock`) 실패가 최종 판정이다(스왑-차감 사이 시점 차 대비)

- 장바구니는 주문 생성이 트랜잭션 내에서 확정된 뒤에만 비운다(실패 시 품목 보존)

트랜잭션 순서

- CART_ITEMS 잠금 조회(FOR UPDATE) — 비어 있으면 400
- 품목별 PRODUCTS 사전 스왑(상품없음/판매중지/재고부족) — 이슈 있으면 409
- OrderService.create 위임: PRODUCTS.stock_qty 차감 → ORDERS 1건 INSERT → ORDER_ITEMS N 건 INSERT (동일 트랜잭션 참여)
- CART_ITEMS 전체 삭제 (주문 확정 후)

사이드이펙트

- 체크아웃 성공 시 품목별로 PRODUCTS.stock_qty가 차감된다(OrderService.create → decreaseStock)
- 체크아웃 성공 시 CART_ITEMS에서 해당 회원의 품목이 전부 삭제된다(deleteAllItems)

오류 응답

코드	사유	발생 조건
400	빈 장바구니	memberId의 장바구니에 품목 없음
400	판매 중인 상품이 아님	OrderService.create 내부 판정 — 사전 스왑 통과 후 시점 차로 발생 가능
404	회원 없음	memberId로 조회된 회원 없음
409	재고 부족/판매중지/상품없음	사전 스왑에서 품목별 사유 목록으로 응답(가용재고/요청수량 포함)
409	재고 부족(차감 실패)	OrderService.create의 decreaseStock 0건 — 스왑 이후 경합

참조 테이블

- CART_ITEMS
- MEMBERS
- PRODUCTS
- ORDERS
- ORDER_ITEMS

curl 예시

```
curl -X POST /api/cart/checkout \
  -H "Content-Type: application/json" \
  -d '{"memberId": "M001"}'
```

INF-ORD-015

INF-ORD-015: GET /api/orders/export — 주문 목록 CSV 내보내기

개요: [[INF-ORD-003]]과 동일한 필터·정렬·상시필터로 조회한 주문 목록을 CSV 파일로 내려받는다. (linked_func: FUNC-order-013, LAB-104)

근거 소스: src/main/java/com/sm/lab/order/controller/OrderController.java:68-79

요청

- Method: GET
- Path: /api/orders/export
- Content-Type: application/json

파라미터	위치	타입	필수	설명
memberId	Query	string	N	회원 ID 필터
orderState	Query	string	N	주문상태 필터(정본)
status	Query	string	N	주문상태 필터 별칭 — orderState와 동시에 오면 orderState 우선(D11, QA r1 FAIL 정정)

응답 (200 OK)

Content-Type: text/csv; charset=UTF-8, Content-Disposition: attachment; filename="orders_{yyyyMMdd_HHmss}.csv"

UTF-8 BOM(EF BB BF)으로 시작하는 CSV 바이너리. 헤더 행 + [[INF-ORD-003]] 목록 조회 결과를 행 단위로 직렬화(페이징 미적용 — 전체 건).

```
orderNo,memberId,status,totalAmount,orderedAt
2026082300001-0101,M0001,PLACED,15000,2026-08-23T10:00:00
```

비즈니스 규칙

- 조회 규칙은 [[INF-ORD-003]]과 완전히 동일(OrderDao.selectOrders 재사용) — 별도 조회 규칙 없음. del_yn = 'N' 상시필터 포함

- 페이징 미적용: offset=0, size=Integer.MAX_VALUE로 고정 호출(전체 건 CSV화)
- orderedAt은 ISO_LOCAL_DATE_TIME 고정 패턴으로 직렬화(가변폭 금지)
- CSV 필드는 RFC 4180 인용 규칙 적용(콤마·큰따옴표·개행 포함 시 큰따옴표로 감싸고 내부 큰따옴표는 이중화)
- CSV 수식 인젝션 방어: 필드 선두 문자가 =,+,-,@이면 작은따옴표(')를 앞에 붙여 Excel이 수식으로 해석하지 않게 함(D11)
- 파일 본문은 UTF-8 BOM 프리픽스 포함(Excel에서 한글 깨짐 방지)

오류 응답

코드	사유	발생 조건
400	유효성 실패	파라미터 타입 오류
401	인증 실패	토큰 없음/만료

인증·인가 (SR-204 — 전역 정책)

이 오류 계약은 /api/** 전역에 적용되는 횡단 정책이다(구현: ApiKeyAuthFilter). 화면 라우트(/product/·/cart·/order/)는 무인증 유지. 개별 엔드포인트 INF는 이 정책을 상속한다.

조건	상태	본문
X-API-Key 미제시/무효	401	{"error": "unauthorized"}
member 스코프 키로 타 회원 자원 접근(소유권 불일치)	403	{"error": "forbidden"}
회원 목록·주문 export 등 관리 자원에 member 키	403	admin 키 전용

- 키 매핑은 앱 설정 lab.api-keys(admin→*, member→회원ID). 경로 판정은 서블릿 정규화 경로 기준(퍼센트 인코딩·matrix 파라미터 우회 차단, default-deny). 소유권은 자원의 실제 member_id를 DB 대조(orders/{orderNo}) — 토큰 문자열 대조가 아니다(SR-204 D15).

반출 상한 (SR-204 R-3)

- 결과 1000행 초과 → **413** {"error": "payload_too_large", "limit": 1000} (부분 반출 위장 없이 전량 거부, CSV 조립 전 판정). 이 엔드포인트는 **admin 키 전용**.

참조 테이블

- ORDERS
- MEMBERS

curl 예시

```
curl -X GET "/api/orders/export?memberId=M0001&orderState=PLACED" \  
-o orders.csv
```

데이터 설계서

SCH-ORD-001 members

SCH-ORD-001: members

FUNC-ID: [TBD] | SRS-F: [TBD] | API: [TBD] | 화면: [TBD]

근거 소스: DB 실측(db-main MCP describe) — 컬럼 타입·NULL·기본값은 MariaDB(sl_lab) 실스키마 조회로 사실화(P1/SR-202 enrichment). [미확인] 헤더 잔존은 자기모순이라 정정(OBS-018, 2026-08-23)

컬럼 설명

컬럼명	타입	NULL	키	기본값	설명
member_id	VARCHAR(20)	N	PK	—	회원 ID (M-접두)
member_name	VARCHAR(50)	N		—	회원명
grade	VARCHAR(10)	N		BRONZE	등급 (BRONZE/SILVER/GOLD/VIP)
phone	VARCHAR(20)	Y		NULL	휴대폰
del_yn	CHAR(1)	N		N	탈퇴 여부 (soft delete)
created_at	DATETIME	N		CURRENT_TIMESTAMP	가입일시

인덱스

인덱스명	컬럼	타입	목적
—	—	—	—

코드값

grade (등급)

| 값 | 의미 | 비고 |

|----|-----|-----|

| BRONZE | 일반 회원 | INF-ORD-001 응답 예시 |

| SILVER | 실버 회원 | INF-ORD-001 응답 예시 |

| GOLD | 골드 회원 | INF-ORD-001 응답 예시 |

| VIP | VIP 회원 | INF-ORD-001 응답 예시 |

관계 (FK / 관찰된 조인)

자식 컬럼	참조 테이블	참조 컬럼	출처	ON DELETE
member_id	ORDERS	member_id	쿼리관찰(4)	—

출처 DB FK=DB 선언 제약, 쿼리관찰(N)=소스 SQL에서 N회 등장한 등가조인(논리 FK).

▶  쿼리 작성 가이드 — 상시 필터 (관찰 1건)

mini-ERD

```
erDiagram
    members {
        varchar(20) member_id PK
        varchar(50) member_name
        varchar(10) grade
        varchar(20) phone
        char(1) del_yn
        datetime created_at
    }
    orders {
        varchar(20) order_no PK
        varchar(20) member_id FK
    }
    members ||--o{ orders : "has"
```

비즈니스 주의사항

- 탈퇴 회원(del_yn = 'Y')은 조회 시 모든 API에서 자동으로 필터됨([[INF-ORD-001]], [[INF-ORD-002]])
- 모든 조회 쿼리에서 del_yn = 'N' 상시필터 적용 — 누락 시 탈퇴 회원도 노출

SCH-ORD-002 order_delivery

SCH-ORD-002: order_delivery

FUNC-ID: [TBD] | SRS-F: [TBD] | API: [TBD] | 화면: [TBD]

근거 소스: DB 실측(db-main MCP describe) — 컬럼 타입·NULL·기본값은 MariaDB(sl_lab) 실스키마 조회로 사실화(P1/SR-202 enrichment). [미확인] 헤더 잔존은 자기모순이라 정정(OBS-018, 2026-08-23)

컬럼 설명

컬럼명	타입	NULL	키	기본값	설명
delivery_no	VARCHAR(20)	N	PK	—	배송번호
order_no	VARCHAR(20)	N		—	주문번호
delivery_state	VARCHAR(20)	N		READY	배송 상태 (READY/SHIPPED/DELIVERED/CANCELED)
invoice_no	VARCHAR(30)	Y		NULL	송장번호
shipped_at	DATETIME	Y		NULL	출고일시

인덱스

인덱스명	컬럼	타입	목적
—	—	—	—

코드값

delivery_state (배송 상태)

값	의미	비고
READY	배송 준비	초기 상태, 송장 미할당
SHIPPED	배송중	송장번호 할당, shipped_at 기록
DELIVERED	배송 완료	고객 수령
CANCELED	배송 취소	주문 취소 시

관계 (FK / 관찰된 조인)

자식 컬럼	참조 테이블	참조 컬럼	출처	ON DELETE
order_no	ORDERS	order_no	INF-ORD-004	—

출처 DB FK=DB 선언 제약, 쿼리관찰(N)=소스 SQL에서 N회 등장한 등가조인(논리 FK).

mini-ERD

```
erDiagram
    orders {
        varchar(20) order_no PK
    }
    order_delivery {
        varchar(20) delivery_no PK
        varchar(20) order_no FK
        varchar(20) delivery_state
        varchar(30) invoice_no
        datetime shipped_at
    }
    orders ||--o{ order_delivery : "ships"
```

비즈니스 주의사항

- 배송은 주문 생성 후 자동으로 생성되지 않음([INF-ORD-004] 응답에서 deliveries는 존재하지 않는 경우가 대부분)
- delivery_state = 'READY' 일 때는 invoice_no와 shipped_at이 NULL
- 배송이 시작되면 invoice_no 할당 및 shipped_at 기록 — AIDD 쿼리 생성 시 shipped_at NULL 체크 필요

SCH-ORD-003 order_items

SCH-ORD-003: order_items

FUNC-ID: [TBD] | SRS-F: [TBD] | API: [TBD] | 화면: [TBD]

근거 소스: DB 실측(db-main MCP describe) — 컬럼 타입·NULL·기본값은 MariaDB(sl_lab) 실스키마 조회로 사실화(P1/SR-202 enrichment). [미확인] 헤더 잔존은 자기모순이라 정정(OBS-018, 2026-08-23)

컬럼 설명

컬럼명	타입	NULL	키	기본값	설명
order_no	VARCHAR(20)	N	PK	—	주문번호
line_no	INT(11)	N	PK	—	주문 라인 번호
sku	VARCHAR(20)	N		—	상품 SKU
qty	INT(11)	N		—	수량
unit_price	BIGINT(20)	N		—	주문 시점 단가(원)

인덱스

인덱스명	컬럼	타입	목적
—	—	—	—

관계 (FK / 관찰된 조인)

자식 컬럼	참조 테이블	참조 컬럼	출처	ON DELETE
order_no	ORDERS	order_no	INF-ORD-004	—
sku	PRODUCTS	sku	쿼리관찰(2)	—

출처 DB FK=DB 선언 제약, 쿼리관찰(N)=소스 SQL에서 N회 등장한 등가조인(논리 FK).

mini-ERD

```
erDiagram
    orders {
        varchar(20) order_no PK
    }
    order_items {
```

```

    varchar(20) order_no PK
    int line_no PK
    varchar(20) sku FK
    int qty
    bigint unit_price
}
products {
    varchar(20) sku PK
    varchar(100) product_name
}
orders ||--o{ order_items : "contains"
products ||--o{ order_items : "references"

```

비즈니스 주의사항

- 복합 PK: (order_no, line_no) — 같은 주문 내에서 라인 번호로 중복 방지([INF-ORD-004] 응답의 items 배열에서 순서 유지)
- unit_price는 **주문 시점의 상품 가격**을 고정 기록 — 현재 PRODUCTS.price와 다를 수 있음([INF-ORD-005] 트랜잭션 순서에서 "라인별로 PRODUCTS 조회 → 재고 차감")
- 주문 생성 후 라인별 productName 조회는 JOIN PRODUCTS ON ORDER_ITEMS.sku = PRODUCTS.sku([INF-ORD-004])

SCH-ORD-004 orders

SCH-ORD-004: orders

FUNC-ID: [TBD] | SRS-F: [TBD] | API: [TBD] | 화면: [TBD]

근거 소스: DB 실측(db-main MCP describe) — 컬럼 타입·NULL·기본값은 MariaDB(sl_lab) 실스키마 조회로 사실화(P1/SR-202 enrichment). [미확인] 헤더 잔존은 자기모순이라 정정(OBS-018, 2026-08-23)

컬럼 설명

컬럼명	타입	NULL	키	기본값	설명
order_no	VARCHAR(20)	N	PK	—	주문번호 (yyyymmdd+seq)
member_id	VARCHAR(20)	N		—	주문 회원
order_state	VARCHAR(20)	N		PLACED	상태 (PLACED/PAID/SHIPPED/PARTIAL_SHIPPED/CANCELLED/DONE)
total_amount	BIGINT(20)	N		—	주문 총액(원)
del_yn	CHAR(1)	N		N	논리 삭제 (soft delete)
ordered_at	DATETIME	N		CURRENT_TIMESTAMP	주문일시

인덱스

인덱스명	컬럼	타입	목적
—	—	—	—

코드값

order_state (주문 상태)

값	의미	비고
PLACED	주문 접수	신규 주문 초기 상태([INF-ORD-005] 생성 시)
PAID	결제 완료	결제 단계([INF-ORD-003] 상태 필터)

SHIPPED	배송중	전체 아이템 배송 시작
PARTIAL_SHIPPED	부분 배송	일부 아이템만 배송([[INF-ORD-003]] 상태 필터)
CANCELED	주문 취소	취소 완료
DONE	주문 완료	모든 배송 완료

관계 (FK / 관찰된 조인)

자식 컬럼	참조 테이블	참조 컬럼	출처	ON DELETE
member_id	MEMBERS	member_id	쿼리관찰(4)	—

출처 DB FK=DB 선언 제약, 쿼리관찰(N)=소스 SQL에서 N회 등장한 등가조인(논리 FK).

▶  쿼리 작성 가이드 — 상시 필터 (관찰 1건)

mini-ERD

```
erDiagram
    members {
        varchar(20) member_id PK
    }
    orders {
        varchar(20) order_no PK
        varchar(20) member_id FK
        varchar(20) order_state
        bigint total_amount
        char(1) del_yn
        datetime ordered_at
    }
    order_items {
        varchar(20) order_no FK
        int line_no PK
    }
    members ||--o{ orders : "places"
    orders ||--o{ order_items : "contains"
```

비즈니스 주의사항

- 모든 조회 쿼리에서 del_yn = 'N' 상시필터 적용([[INF-ORD-003]], [[INF-ORD-004]]) — 논리삭제된 주문 조회 시 404
- order_no 채번: yyyyMMdd-{4자리 시퀀스} 형식, 시작값 100([[INF-ORD-005]]) — AIDD 쿼리 생성 시 시퀀스 직접 UPDATE 대신 애플리케이션 로직에 의존
- total_amount는 각 라인 unit_price × qty의 합([[INF-ORD-005]]) — DB에서 직접 계산하지 말고 애플리케이션에서 계산해 INSERT
- member_id 조인 시 회원이 탈퇴했어도 주문 기록은 유지([[INF-ORD-003]] "회원 미탈퇴 여부와 무관하게 조인만 수행")

SCH-ORD-005 products

SCH-ORD-005: products

FUNC-ID: [TBD] | SRS-F: [TBD] | API: [TBD] | 화면: [TBD]

근거 소스: DB 실측(db-main MCP describe) — 컬럼 타입·NULL·기본값은 MariaDB(sl_lab) 실스키마 조회로 사실화(P1/SR-202 enrichment). [미확인] 헤더 잔존은 자기모순이라 정정(OBS-018, 2026-08-23)

컬럼 설명

컬럼명	타입	NULL	키	기본값	설명
sku	VARCHAR(20)	N	PK	—	상품 SKU
product_name	VARCHAR(100)	N		—	상품명
price	BIGINT(20)	N		—	판매가(원)
stock_qty	INT(11)	N		0	가용 재고
sale_yn	CHAR(1)	N		Y	판매 여부

인덱스

인덱스명	컬럼	타입	목적
—	—	—	—

코드값

sale_yn (판매 여부)

값	의미	비고
Y	판매중	주문 생성 시 필수([INF-ORD-005]) 검증 "sale_yn = 'Y'가 아니면 400"
N	판매 중단	주문 생성 불가

관계 (FK / 관찰된 조인)

자식 컬럼	참조 테이블	참조 컬럼	출처	ON DELETE
sku	ORDER_ITEMS	sku	쿼리관찰(2)	—

출처 DB FK=DB 선언 제약, 쿼리관찰(N)=소스 SQL에서 N회 등장한 등가조인(논리 FK).

▶ 쿼리 작성 가이드 — 상시 필터 (관찰 1건)

mini-ERD

```
erDiagram
    products {
        varchar(20) sku PK
        varchar(100) product_name
        bigint price
        int stock_qty
        char(1) sale_yn
    }
    order_items {
        varchar(20) sku FK
        int qty
        bigint unit_price
    }
    products ||--o{ order_items : "has"
```

비즈니스 주의사항

- 주문 생성 시 상품 검증: sale_yn = 'Y'가 아니면 400 오류([INF-ORD-005]) — AIDD 쿼리 생성 시 필수 필터
- 재고 차감: 조건부 UPDATE WHERE stock_qty >= qty 사용 — 동시 주문 시 데이터베이스 레벨 동시성 가드([INF-ORD-005] "재고 부족으로 409")
- price는 현재 판매가 — ORDER_ITEMS.unit_price는 주문 시점의 가격을 별도 기록([INF-ORD-004] 상세 조회 시 과거 단가 표시)
- stock_qty UPDATE는 AIDD 쿼리 생성 시 직접 위임 안 함 — 동시성 문제 방지 위해 애플리케이션이 주도

SCH-ORD-006 cart_items

SCH-ORD-006: cart_items

FUNC-ID: [TBD] | SRS-F: [TBD] | API: [INF-ORD-010](#), [INF-ORD-011](#), [INF-ORD-012](#), [INF-ORD-013](#) | 화면: [TBD]

근거 소스: src/main/java/com/sm/lab/order/controller/CartController.java +
src/main/resources/mapper/cart.xml

컬럼 설명

컬럼명	타입	NULL	키	기본값	설명
member_id	VARCHAR(20)	N	PK	—	회원 ID (M-접두)
sku	VARCHAR(20)	N	PK	—	상품 SKU
qty	INT(11)	N		—	장바구니 수량
added_at	DATETIME	N		CURRENT_TIMESTAMP	장바구니 담은 날짜시간

인덱스

인덱스명	컬럼	타입	목적
idx_cart_items_added_at	added_at	—	담은 순 정렬([[INF-ORD-011]] 응답에서 added_at, sku 순 정렬)

관계 (FK / 관찰된 조인)

자식 컬럼	참조 테이블	참조 컬럼	출처	ON DELETE
member_id	MEMBERS	member_id	INF-ORD-010	—
sku	PRODUCTS	sku	쿼리관찰(4)	—

출처 DB FK=DB 선언 제약, 쿼리관찰(N)=소스 SQL에서 N회 등장한 등가조인(논리 FK).

mini-ERD

```
erDiagram
    members {
        varchar(20) member_id PK
    }
    products {
```

```

    varchar(20) sku PK
    varchar(100) product_name
    bigint price
    int stock_qty
}
cart_items {
    varchar(20) member_id PK
    varchar(20) sku PK
    int qty
    datetime added_at
}
members ||--o{ cart_items : "has"
products ||--o{ cart_items : "referenced"

```

비즈니스 주의사항

- 복합 PK: (member_id, sku) — 같은 상품의 재담기는 UPSERT로 수량 합산([[INF-ORD-010]] "INSERT ... ON DUPLICATE KEY UPDATE qty = qty + VALUES(qty)")
- 응답의 productName, price, lineTotal은 테이블 컬럼이 아니라 애플리케이션이 JOIN PRODUCTS로 조회 후 계산해서 제공([[INF-ORD-011]] "totalAmount는 각 품목의 price × qty 합계로, 조회 시점에 애플리케이션에서 계산")
- 판매중지(PRODUCTS.sale_yn='N') 상품은 담기 거부(409 — [[INF-ORD-010]])
- 품절(PRODUCTS.stock_qty=0)은 담기 전 조기 거부(409 — [[INF-ORD-010]])
- 담기는 상품 재고를 직접 차감하지 않음([[INF-ORD-010]] "담기는 재고를 차감하지 않는다(보관 전용)")
- 품목 삭제는 명시적 DELETE만 가능, qty를 0으로 변경할 수 없음([[INF-ORD-012]] "qty < 1은 400으로 거부한다 — 삭제는 명시적 DELETE로만 수행")